3. Accuracy (`'error'`) - Accuracy is the percent of correct predictions. XGBoost uses the error, which is one minus accuracy because it wants to minimize the value.

4. You can also create a custom function for evaluation.

In the following example, I set the `eval_metric` to `'error'`.

```
>>> xg_err = xgb.XGBClassifier(early_stopping_rounds=20,
...                            eval_metric='error')
>>> xg_err.fit(X_train, y_train,
...        eval_set=[(X_train, y_train),
...                  (X_test, y_test)
...                 ]
...       )
>>> xg_err.score(X_test, y_test)
[0] validation_0-error:0.24739  validation_1-error:0.27072
[1] validation_0-error:0.24218  validation_1-error:0.26188
[2] validation_0-error:0.23839  validation_1-error:0.24751
[3] validation_0-error:0.23697  validation_1-error:0.25193
[4] validation_0-error:0.23081  validation_1-error:0.24530
[5] validation_0-error:0.22607  validation_1-error:0.24420
[6] validation_0-error:0.22180  validation_1-error:0.24862
[7] validation_0-error:0.21801  validation_1-error:0.24862
[8] validation_0-error:0.21280  validation_1-error:0.25304
[9] validation_0-error:0.21043  validation_1-error:0.25304
[10]    validation_0-error:0.20806  validation_1-error:0.24641
[11]    validation_0-error:0.20284  validation_1-error:0.25193
[12]    validation_0-error:0.20047  validation_1-error:0.24420
[13]    validation_0-error:0.19668  validation_1-error:0.24420
[14]    validation_0-error:0.19384  validation_1-error:0.24530
[15]    validation_0-error:0.18815  validation_1-error:0.24199
[16]    validation_0-error:0.18531  validation_1-error:0.24199
[17]    validation_0-error:0.18389  validation_1-error:0.23867
[18]    validation_0-error:0.18531  validation_1-error:0.23757
[19]    validation_0-error:0.18815  validation_1-error:0.23867
[20]    validation_0-error:0.18246  validation_1-error:0.24199
[21]    validation_0-error:0.17915  validation_1-error:0.24862
[22]    validation_0-error:0.17867  validation_1-error:0.24751
[23]    validation_0-error:0.17630  validation_1-error:0.24199
[24]    validation_0-error:0.17488  validation_1-error:0.24309
[25]    validation_0-error:0.17251  validation_1-error:0.24530
[26]    validation_0-error:0.17204  validation_1-error:0.24309
[27]    validation_0-error:0.16825  validation_1-error:0.24199
[28]    validation_0-error:0.16730  validation_1-error:0.24088
[29]    validation_0-error:0.16019  validation_1-error:0.24199
[30]    validation_0-error:0.15782  validation_1-error:0.24972
[31]    validation_0-error:0.15972  validation_1-error:0.24862
[32]    validation_0-error:0.15924  validation_1-error:0.24641
[33]    validation_0-error:0.15403  validation_1-error:0.25635
[34]    validation_0-error:0.15261  validation_1-error:0.25525
[35]    validation_0-error:0.15213  validation_1-error:0.25525
[36]    validation_0-error:0.15166  validation_1-error:0.25525
```

```
[37]    validation_0-error:0.14550  validation_1-error:0.25525
[38]    validation_0-error:0.14597  validation_1-error:0.25083
0.7624309392265194
```

Now, the best number of trees when we minimize error is 19.

```
>>> xg_err.best_ntree_limit
19
```

10.4 Summary

You can use the `early_stopping_rounds` parameter in XGBoost as a feature to specify a maximum number of trees after the lowest evaluation metric is encountered. This is a useful feature to avoid overfitting, as it allows you to specify a validation set and track the model's performance. If the performance hasn't improved after a certain number of rounds (specified by `early_stopping_rounds`), then training is stopped, and the best model is kept. This helps to avoid wasting time and resources on training a model that is unlikely to improve.

Another benefit of the early stopping rounds parameter is that it allows you to select the best model with the least training time. In other words, it's like getting to the finish line first without running the entire race!

10.5 Exercises

1. What is the purpose of early stopping in XGBoost?
2. How is the `early_stopping` parameter used in XGBoost?
3. How does the `eval_set` parameter work with the early stopping round parameter in XGBoost?

Chapter 11

XGBoost Hyperparameters

XGBoost provides relatively good performance out of the box. However, it is also a complicated model with many knobs and dials. You can adjust these dials to improve the results. This chapter will introduce many of these knobs and dials, called hyperparameters.

11.1 Hyperparameters

What does the term *hyperparameter* even mean? In programming, a parameter lets us control how a function behaves. Hyperparameters allow us to control how a machine-learning model behaves.

The scikit-learn library has some basic conventions for hyperparameters that allow us to think of them as levers that push a model towards overfitting or underfitting. One convention is that hyperparameters that start with `max_` will tend to complicate the model (and lead to overfitting) when you raise the value. Conversely, they will simplify the model if you lower them (and lead to underfitting). Often there is a sweet spot in between.

We saw n example of this earlier when we looked at decision trees. Raising the `max_depth` hyperparameter will add child nodes, complicating the model. Lowering it will simplify the model (degrading it into a decision stump).

There is a similar convention for parameters starting with `min_`. Those tend to simplify when you raise them and complicate when you lower them.

I will split up the parameters by what they impact. There are parameters for tree creation, sampling, categorical data, ensembling, regularization, and imbalanced data.

Here are the tree hyperparameters:

- `max_depth=6` - Tree depth. How many feature interactions you can have. Larger is more complex (more likely to overfit). Each level doubles the time. Range [0, ∞].
- `max_leaves=0` - Number of leaves in a tree. A larger number is more complex. (Not supported by `tree_method='exact'`). Range [0, ∞].
- `min_child_weight=1` - Minimum sum of hessian (weight) needed in a child. The larger the value, the more conservative. Range [0,∞]
- `'grow_policy="depthwise"'` - (Only supported with `tree_method` set to `'hist'`, `'approx'`, or `'gpu_hist'`. Split nodes closest to root. Set to `"lossguide"` (with `max_depth=0`) to mimic LightGBM behavior.
- `tree_method="auto"` - Use `"hist"` to use histogram bucketing and increase performance. `"auto"` heuristic:
  - `'exact'` for small data
  - `'approx'` for larger data
  - `'hist'` for histograms (limit bins with `max_bins` (default 256))

These are the sampling hyperparameters. These work cumulatively. First, tree, then level, then node. If `colsample_bytree`, `colsample_bylevel`, and `colsample_bynode` are all .5, then a tree will only look at (.5 * .5 * .5) 12.5% of the original columns.

- `colsample_bytree=1` - Subsample columns at each tree. Range (0,1]. Recommended to search [.5, 1].
- `colsample_bylevel=1` - Subsample columns at each level (from tree columns `colsample_bytree`). Range (0,1]. Recommended to search [.5, 1].
- `colsample_bynode=1` - Subsample columns at each node split (from tree columns `colsample_bytree`). Range (0,1]. Recommended to search [.5, 1].
- `subsample=1` - Sample portion of training data (rows) before each boosting round. Range (0,1]. Lower to make more conservative. (When not equal to 1.0, the model performs *stochastic gradient descent*, i.e., there is some randomness in the model.) Recommended to search [.5, 1].
- `sampling_method='uniform'` - Sampling method. `'uniform'` for equal probability. `gradient_based` sample is proportional to the regularized absolute value of gradients (only supported by `tree_method="gpu_hist"`.

Hyperparameters for managing categorical data(use `enable_categorical=True` parameter and set columns to Pandas categoricals `.astype('category')`).

- `max_cat_to_onehot=4` - Upper threshold for using one-hot encoding for categorical features. One-hot encoding is used when the number of categories is less than this number.
- `max_cat_threshold=64` - Maximum number of categories to consider for each split.

Ensembling hyperparameters control how the subsequent trees are created:

- `n_estimators=100` - Number of trees. Larger is more complex. Default 100. Use `early_stopping_rounds` to prevent overfitting. You don't really need to tune if you use `early_stopping_rounds`.
- `early_stopping_rounds=None` - Stop creating new trees if `eval_metric` score has not improved after n rounds. Range [1,∞]
- `eval_metric` - Metric for evaluating validation data for evaluating early stopping.
  - Classification metrics
    - `'logloss'` - Default classification metric. Negative log-likelihood.
    - `'auc'` - Area under the curve of Receiver Operating Characteristic.
- `objective` - Learning objective to optimize the fit of the model to data during training.
  - Classification objectives
    - `'binary:logistic'` - Default classification objective. Outputs probability.
    - `'multi:softmax'` - Multi-class classification objective.

Regularization hyperparameters control the complexity of the overall model:

- `learning_rate=.3` - After each boosting round, multiply weights to make it more conservative. Lower is more conservative. Range (0,1]. Typically when you lower this, you want to raise `n_estimators`. Range (0, 1]
- `gamma=0` / `min_split_loss` - L0 regularization. Prunes tree to remove splits that don't meet the given value. Global regularization. Minimum loss required for making a split. Larger is more conservative. Range [0, ∞), default 0 - no regularization. Recommended search is (0, 1, 10, 100, 1000…)

- `reg_alpha=0` - L1/ridge regularization. (Mean of weights). Increase to be more conservative. Range [0, ∞)
- `reg_lambda=0` - L2 regularization. (Root of squared weights). Increase to be more conservative. Range [0, ∞)

Imbalanced data hyperparameters:

- `scale_pos_weight=1` - Consider (count negative) / (count positive) for imbalanced classes. Range (0, large number)
- `max_delta_step=0` - Maximum delta step for leaf output. It might help with imbalanced classes. Range [0, ∞)
- Use `'auc'` or `'aucpr'` for `eval_metric` metric (rather than classification default `'logless'`)

11.2 Examining Hyperparameters

You can set hyperparameters by passing them into the constructor when creating an XGBoost model. If you want to inspect hyperparameters, you can use the `.get_params` method:

```
>>> xg = xgb.XGBClassifier() # set the hyperparamters in here
>>> xg.fit(X_train, y_train)
>>> xg.get_params()
{'objective': 'binary:logistic',
 'use_label_encoder': None,
 'base_score': 0.5,
 'booster': 'gbtree',
 'callbacks': None,
 'colsample_bylevel': 1,
 'colsample_bynode': 1,
 'colsample_bytree': 1,
 'early_stopping_rounds': None,
 'enable_categorical': False,
 'eval_metric': None,
 'feature_types': None,
 'gamma': 0,
 'gpu_id': -1,
 'grow_policy': 'depthwise',
 'importance_type': None,
 'interaction_constraints': '',
 'learning_rate': 0.300000012,
 'max_bin': 256,
 'max_cat_threshold': 64,
 'max_cat_to_onehot': 4,
 'max_delta_step': 0,
 'max_depth': 6,
 'max_leaves': 0,
 'min_child_weight': 1,
 'missing': nan,
 'monotone_constraints': '()',
 'n_estimators': 100,
 'n_jobs': 0,
 'num_parallel_tree': 1,
 'predictor': 'auto',
```

```
 'random_state': 0,
 'reg_alpha': 0,
 'reg_lambda': 1,
 'sampling_method': 'uniform',
 'scale_pos_weight': 1,
 'subsample': 1,
 'tree_method': 'exact',
 'validate_parameters': 1,
 'verbosity': None}
```

This shows the default valus for each hyperparameter.

11.3 Tuning Hyperparameters

Hyperparameters can be a bit tricky to tune. Let's explore how to tune a single value first.

To get started, you'll want to use validation curves. These plots show the relationships between your model's performance and the values of specific hyperparameters. They show the values of hyperparameters that yield the best results.

I like to plot a validation curve by using the `validation_curve` function from the Yellowbrick library.

Let's examine how the *gamma* parameter impacts our model. You can think of gamma as a pruner for the trees. It limits growth unless a specified loss reduction is met. The larger the value, the more simple (conservative or tending to underfit) our model.

```
fig, ax = plt.subplots(figsize=(8, 4))
ms.validation_curve(xgb.XGBClassifier(), X_train, y_train, param_name='gamma',
    param_range=[0, .5, 1,5,10, 20, 30], n_jobs=-1, ax=ax)
```


It looks like gamma is best between 10 and 20. We could run another validation curve to dive into that area. However, we will soon see another mechanism for finding a good value without requiring us to specify every single option we want to consider.

11.4 Intuitive Understanding of Learning Rate

Returning to our golf metaphor, the learning rage hyperparameter is like how hard you swing. It's all about finding the right balance between caution and aggression. Just like in golf, if you swing too hard, you can end up in the rough. But if you swing too softly, you'll never get the ball in the hole.

With XGBoost, if you set the `learning_rate` hyperparameter too high, your model will be over-confident and over-fit the data. On the other hand, if you set `learning_rate` too low, your model will take too long to learn and never reach its full potential.

We want to find the right balance that maximizes your model's accuracy. You'll have to experiment to find out where that sweet spot is. In golf – you experiment with different techniques, adjust your swing and your grip, and it'll take some practice, but eventually, you might find the perfect combination that will allow you to hit a hole-in-one.

Let's go back to our model with two layers of depth and examine the values in the leaf node when the `learning_rate` is set to 1.

```
# check impact of learning weight on scores
xg_lr1 = xgb.XGBClassifier(learning_rate=1, max_depth=2)
xg_lr1.fit(X_train, y_train)
```

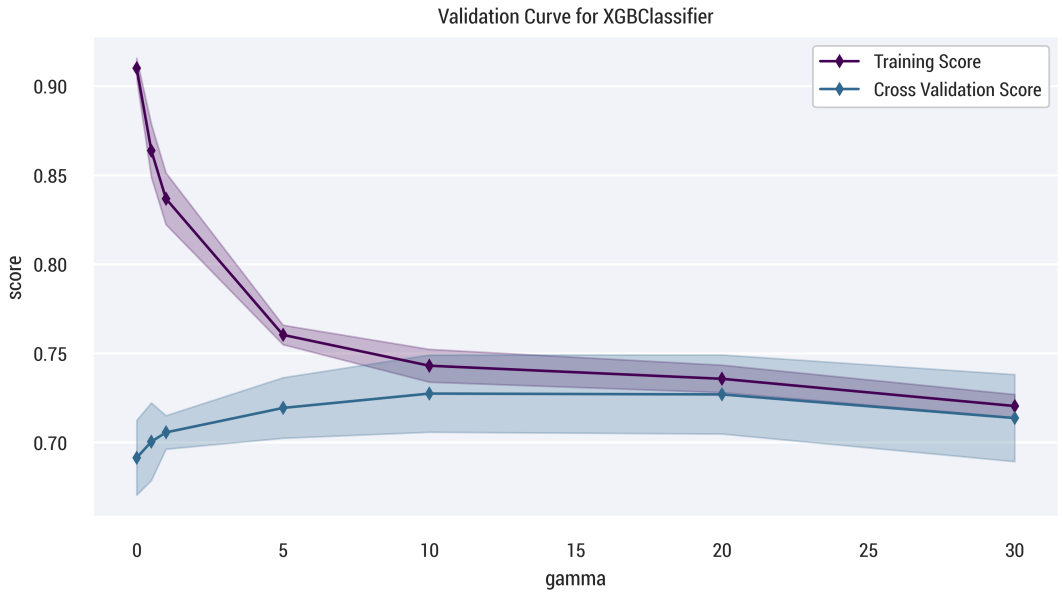


Figure 11.1: Validation curve for the gamma hyperparameter. Around 10-20 the score is maximized for the cross validation line. The cross validation line gives us a feel for how the model will perform on unseen data, hwile the training score helps us to understand if the model is overfitting.

```
my_dot_export(xg_lr1, num_trees=0, filename='img/xg_depth2_tree0.dot',
              title='Learning Rate set to 1')
```

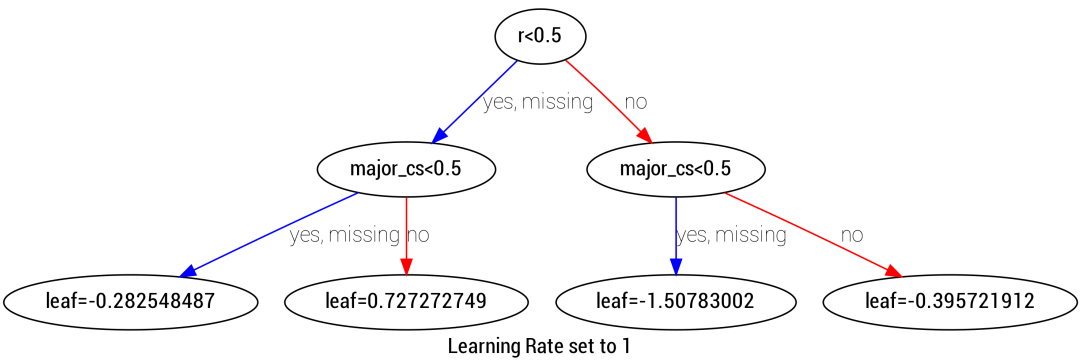


Figure 11.2: When the learning rate is set to one, we preserve the log loss score.

Now we are going to decrease the learning rate. Remember, this is like being really timid with our golf swing.

```
# check impact of learning weight on scores
xg_lr001 = xgb.XGBClassifier(learning_rate=.001, max_depth=2)
xg_lr001.fit(X_train, y_train)
```

```
my_dot_export(xg_lr001, num_trees=0, filename='img/xg_depth2_tree0_lr001.dot',
              title='Learning Rate set to .001')
```

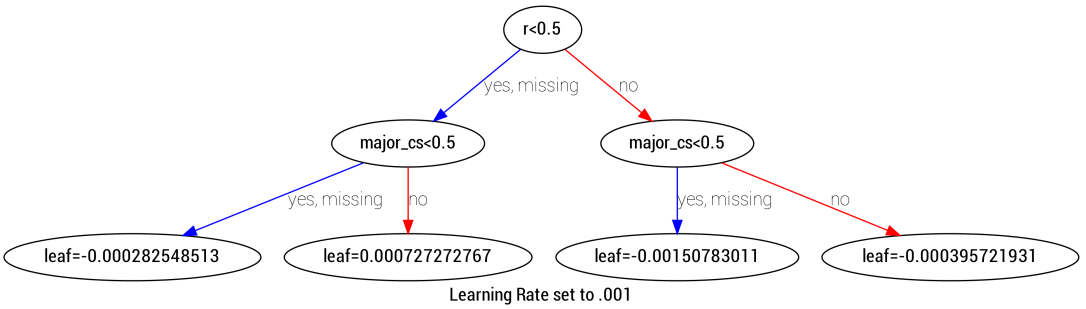


Figure 11.3: Lowering the learning rate essentially multiples the full values by this amount in the first tree. Each subsequent tree is now impacted and will shift.

You can see that the learning rate took the values in the leaves and decreased them. Positive values will still push toward the positive label but at a slower rate. We could also set the learning rate to a value above 1, but it does not make sense in practice. It is like teeing off with your largest club when playing miniature golf. You can try it, but you will see that the model does not perform well.

My suggestion for tuning the learning rate (as you will see when I show step-wise tuning) is to tune it last. Combine this with early stopping and a large number of trees. If early stopping doesn't kick in, raise the number of trees (it means your model is still improving).

11.5 Grid Search

Calibrating one hyperparameter takes some time, there are multiple hyperparameters, some of which interact with other hyperparameters. How do we know which combination is best? We can try various combinations and explore which values perform the best.

The `GridSerachCV` class from scikit-learn will do this for us. We need to provide it with a mapping of hyperparameters to potential values for each hyperparameter. It will loop through all combinations and keep track of the best ones. If you have imbalanced data or need to optimize for a different metric, you can also set the `scoring` parameter.

Be careful with grid search. Running grid search on XGBoost can be computationally expensive, as it can take a long time for XGBoost to run through every combination of hyperparameters. It can also be difficult to determine how many hyperparameters to test and which values to use as candidates.

Later, I will show another method I prefer more than grid search for tuning the model.

This example below runs quickly on my machine, but it isn't testing that many combinations. You could imagine (ignoring `random_state`) if any hyperparameters had ten options, there would be 1,000,000 tests to run.

```
from sklearn import model_selection
params = {'reg_lambda': [0],  # No effect
          'learning_rate': [.1, .3], # makes each boost more conservative
          'subsample': [.7, 1],
          'max_depth': [2, 3],
          'random_state': [42],
```

```
          'n_jobs': [-1],
          'n_estimators': [200]}

xgb2 = xgb.XGBClassifier(early_stopping_rounds=5)
cv = (model_selection.GridSearchCV(xgb2, params, cv=3, n_jobs=-1)
    .fit(X_train, y_train,
         eval_set=[(X_test, y_test)],
         verbose=50
    )
)
```

After running the grid search, we can inspect the `.best_params_` attribute. This provides a dictionary of the hyperparameters with the best result.

Here are the best hyperparameters from our minimal grid search:

```
>>> cv.best_params_
{'learning_rate': 0.3,
 'max_depth': 2,
 'n_estimators': 200,
 'n_jobs': -1,
 'random_state': 42,
 'reg_lambda': 0,
 'subsample': 1}
```

Let's stick those values in a dictionary, and then we can use *dictionary unpacking* to provide those values as arguments when we create the model. The `verbose=10` parameter tells XGBoost only to provide training metrics after every ten attempts. We will use `.fit` to train the model.

```
params = {'learning_rate': 0.3,
          'max_depth': 2,
          'n_estimators': 200,
          'n_jobs': -1,
          'random_state': 42,
          'reg_lambda': 0,
          'subsample': 1
}
xgb_grid = xgb.XGBClassifier(**params, early_stopping_rounds=50)
xgb_grid.fit(X_train, y_train, eval_set=[(X_train, y_train),
 (X_test, y_test)],
 verbose=10
)
```

Now let's train a default model to compare if there is an improvement from using grid search.

```
# vs default
xgb_def = xgb.XGBClassifier(early_stopping_rounds=50)
xgb_def.fit(X_train, y_train, eval_set=[(X_train, y_train),
 (X_test, y_test)],
 verbose=10
)
```

Here's the accuracy for both the default and the grid models:

```
>>> xgb_def.score(X_test, y_test), xgb_grid.score(X_test, y_test)
(0.7558011049723757, 0.7524861878453039)
```

Oddly enough, it looks like the default model is slightly better. This is a case where you need to be careful. The grid search did a 3-fold cross-validation to find the hyperparameters, but the final score was calculated against a single holdout set.

K-fold cross-validation (in the grid search case, k was 3) splits your data set into k *folds* or groups. Then, it reserves a fold to test the model and trains it on the other data. It repeats this for each fold. Finally, it averages out the results to create a more accurate prediction. It's a great way to understand better how your model will perform in the real world.

When you do k-fold validation, you may get different results than just calling `.fit` on a model and `.score`. To run k-fold outside of grid search, you can use the `cross_val_score` function from scikit-learn.

Below, we run cross-validation on the default model:

```
>>> results_default = model_selection.cross_val_score(
...     xgb.XGBClassifier(),
...     X=X, y=y,
...     cv=4
... )
```

Here are the scores from each of the four folds:

```
>>> results_default
array([0.71352785, 0.72413793, 0.69496021, 0.74501992])
```

Some folds performed quite a bit better than others. Our average score (accuracy) is 72%.

```
>>> results_default.mean()
0.7194114787534214
```

Let's call `cross_val_score` again with a model created from our grid search hyperparameters.

```
>>> results_grid = model_selection.cross_val_score(
...     xgb.XGBClassifier(**params),
...     X=X, y=y,
...     cv=4
... )
```

Here are the fold scores. Note that these values do not deviate as much as the default model.

```
>>> results_grid
array([0.74137931, 0.74137931, 0.74801061, 0.73572377])
```

Here is the mean score:

```
>>> results_grid.mean()
0.7416232505873941
```

Our grid search model did perform better (at least for these folds of the data). And the scores across the folds were more consistent. It is probably the case that the test set we held out for `X_test` is easier for the default model to predict.

11.6 Summary

Tuning hyperparameters can feel like a stab in the dark. Validation curves can start us down the path but only work for a single value. You believe there is a sweet spot somewhere, but you have to dig deep and hunt for the other sweet spots of the other hyperparameters. We can combine this with grid search, but it can be tedious.

We also showed how using k-fold validation can help us understand how consistent the model is and whether some parts of the data might be easier to model. You will be more confident in a consistent model.

11.7 Exercises

1. What is the purpose of hyperparameters in XGBoost?
2. How can validation curves be used to tune hyperparameters in XGBoost?
3. What is the impact of the gamma hyperparameter on XGBoost models?
4. How can the learning rate hyperparameter affect the performance of an XGBoost model?
5. How can grid search be used to tune hyperparameters in XGBoost?

Chapter 12

Hyperopt

Remember I said that grid search could be slow? It also only searches over the values that I provide. What if another value is better, but I didn't tell grid search about that possible value? Grid search is naive and would have no way of suggesting that value. Let's look at another library that might help with this but also search in a smarter way than the brute force mechanism of grid search—introducing `hyperopt`.

Hyperopt is a Python library for optimizing both discrete and continuous hyperparameters for XGBoost. Hyperopt uses *Bayesian optimization* to tune hyperparameters. This uses a probabilistic model to select the next set of hyperparameters to try. If one value performs better, it will try values around it and see if they boost the performance. If the values worsen the model, then Hyperopt can ignore those values as future candidates. Hyperopt can also tune various other machine-learning models, including random forests and neural networks.

12.1 Bayesian Optimization

Bayesian optimization algorithms offer several benefits for hyperparameter optimization, including:

- Efficiency: Bayesian optimization algorithms use probabilistic models to update their predictions based on previous trials, which allows them to quickly identify promising areas of the search space and avoid wasteful exploration.
- Accuracy: Bayesian optimization algorithms can incorporate prior knowledge and adapt to the underlying structure of the optimization problem, which can result in more accurate predictions.
- Flexibility: Bayesian optimization algorithms are flexible and can be applied to a wide range of optimization problems, including those with complex and multimodal search spaces.

Using Bayesian optimization algorithms can significantly improve hyperparameter optimization's efficiency, accuracy, and robustness, leading to better models.

12.2 Exhaustive Tuning with Hyperopt

Let's look at an exhaustive exploration with Hyperopt. This code might be confusing if you aren't familiar with the notion of *first-class functions* in Python. Python allows us to pass a function in as a parameter to another function or return a function as a result. To use the Hyperopt library, we need to use the `fmin` function which tries to find the best hyperparameter values given a space of many parameters.

The `fmin` function expects us to pass in another function that accepts a dictionary of hyperparameters to evaluate and returns a dictionary. Our `hyperparameter_tuning` function doesn't quite fit the bill. It accepts a dictionary, `space`, as the first parameter but it has additional parameters. I will use a *closure* in the form of a `lambda` function to adapt the `hyperparameter_tuning` function to a new function that accepts the hyperparameter dictionary.

The `hyperparameter_tuning` function takes in a dictionary of hyperparameters (`space`), training data (`X_train` and `y_train`), test data (`X_test` and `y_test`), and an optional value for early stopping rounds (`early_stopping_rounds`). Some hyperparameters need to be integers, so it converts those keys and adds `early_stopping_rounds` to the hyperparameters to evaluate. Then it trains a model and returns a dictionary with a negative accuracy score and a status.

Because `fmin` tries to *minimize* the score and our model is evaluating the accuracy, we do not want the model with the minimum accuracy. However, we can do a little trick and minimize the negative accuracy.

```
from hyperopt import fmin, tpe, hp, STATUS_OK, Trials
from sklearn.metrics import accuracy_score, roc_auc_score

from typing import Any, Dict, Union

def hyperparameter_tuning(space: Dict[str, Union[float, int]],
                          X_train: pd.DataFrame, y_train: pd.Series,
                          X_test: pd.DataFrame, y_test: pd.Series,
                          early_stopping_rounds: int=50,
                          metric:callable=accuracy_score) -> Dict[str, Any]:
    """
    Perform hyperparameter tuning for an XGBoost classifier.

    This function takes a dictionary of hyperparameters, training
    and test data, and an optional value for early stopping rounds,
    and returns a dictionary with the loss and model resulting from
    the tuning process. The model is trained using the training
    data and evaluated on the test data. The loss is computed as
    the negative of the accuracy score.

    Parameters
    ----------
    space : Dict[str, Union[float, int]]
        A dictionary of hyperparameters for the XGBoost classifier.
    X_train : pd.DataFrame
        The training data.
    y_train : pd.Series
        The training target.
    X_test : pd.DataFrame
        The test data.
    y_test : pd.Series
        The test target.
    early_stopping_rounds : int, optional
        The number of early stopping rounds to use. The default value
        is 50.
    metric : callable
        Metric to maximize. Default is accuracy
```

```
    Returns
    -------
    Dict[str, Any]
        A dictionary with the loss and model resulting from the
        tuning process. The loss is a float, and the model is an
        XGBoost classifier.
    """
    int_vals = ['max_depth', 'reg_alpha']
    space = {k: (int(val) if k in int_vals else val)
             for k,val in space.items()}
    space['early_stopping_rounds'] = early_stopping_rounds
    model = xgb.XGBClassifier(**space)
    evaluation = [(X_train, y_train),
                  (X_test, y_test)]
    model.fit(X_train, y_train,
              eval_set=evaluation,
              verbose=False)

    pred = model.predict(X_test)
    score = metric(y_test, pred)
    return {'loss': -score, 'status': STATUS_OK, 'model': model}
```

After we have defined the function we want to minimize, we need to define the space of the search for our hyperparameters. Based on a talk by Bradley Boehmke[1], I've used these values and found them to work quite well. We will cover the functions that describe the search space in a moment.

Next, we create a `trials` object to store the results of the hyperparameter tuning process. The `fmin` function loops over the options to search for the best hyperparameters. I told it to use the `tpe.suggest` algorithm, which runs the *Tree-structured Parzen Estimator - Expected Improvement* Bayesian Optimization. It uses the expected improvement acquisition function to guide the search. This algorithm works well with noisy data. We also told the search to limit the evaluations to 2,000 attempts. My Macbook Pro (2022) takes about 30 minutes to run this search. On my Thinkpad P1 (2020), it takes over 2 hours. You can set a timeout period as well.

As this runs, it spits out the best loss scores.

```
options = {'max_depth': hp.quniform('max_depth', 1, 8, 1),  # tree
    'min_child_weight': hp.loguniform('min_child_weight', -2, 3),
    'subsample': hp.uniform('subsample', 0.5, 1),   # stochastic
    'colsample_bytree': hp.uniform('colsample_bytree', 0.5, 1),
    'reg_alpha': hp.uniform('reg_alpha', 0, 10),
    'reg_lambda': hp.uniform('reg_lambda', 1, 10),
    'gamma': hp.loguniform('gamma', -10, 10), # regularization
    'learning_rate': hp.loguniform('learning_rate', -7, 0),  # boosting
    'random_state': 42
}

trials = Trials()
best = fmin(fn=lambda space: hyperparameter_tuning(space, X_train, y_train,
```

[1] https://bradleyboehmke.github.io/xgboost_databricks_tuning/index.html#slide21

```
                                                    X_test, y_test),
    space=options,
    algo=tpe.suggest,
    max_evals=2_000,
    trials=trials,
    #timeout=60*5 # 5 minutes
)
```

The best variable holds the results. If I'm using a notebook, I will inspect the values and make sure I keep track of the hyperparameters. I like to print them out and then stick them into a dictionary, so I have them for posterity.

```
# 2 hours of training (paste best in here)
long_params = {'colsample_bytree': 0.6874845219014455,
               'gamma': 0.06936323554883501,
               'learning_rate': 0.21439214284976907,
               'max_depth': 6,
               'min_child_weight': 0.6678357091609912,
               'reg_alpha': 3.2979862933185546,
               'reg_lambda': 7.850943400390477,
               'subsample': 0.999767483950891}
```

Now we can train a model with these hyperparameters.

```
xg_ex = xgb.XGBClassifier(**long_params, early_stopping_rounds=50,
                          n_estimators=500)
xg_ex.fit(X_train, y_train,
       eval_set=[(X_train, y_train),
                 (X_test, y_test)
                ],
        verbose=100
      )
```

How does this model do?

```
>>> xg_ex.score(X_test, y_test)
0.7580110497237569
```

This is an improvement over the default out-of-the-box model. Generally, you will get better improvement in your model by understanding your data and doing appropriate feature engineering than the improvement you will see with hyperparameter optimization. However, the two are not mutually exclusive, and you can and should use both.

12.3 Defining Parameter Distributions

Rather than limiting the hyperparameters to selection from a list of options, the hyperopt library has various functions that let us define how to choose values.

The choice function can specify a discrete set of values for a categorical hyperparameter. This is similar to enumerating the list of options in a grid search. This function takes two arguments: a list of possible values and an optional probability distribution over these values. It returns a random value from the list according to the specified probability distribution.

For example, to generate a random value from a list of possible values ['a', 'b', 'c'], you can use the following code:

```
>>> from hyperopt import hp, pyll
>>> pyll.stochastic.sample(hp.choice('value', ['a', 'b', 'c']))
'a'
```

To generate a random value from a list of possible values `['a', 'b', 'c']` with probabilities `[0.05, 0.9, 0.05]` use the `choice` function:

```
>>> pyll.stochastic.sample(hp.pchoice('value', [(.05, 'a'), (.9, 'b'),
...     (.05, 'c')]))
'c'
```

Note

Be careful about using `choice` and `pchoice` for numeric values. The Hyperopt library treats each value as independent and not ordered. Its search algorithm cannot take advantage of the outcomes of neighboring values. I have found examples of using Hyperopt that suggest defining the search space for `'num_leaves'` and `'subsample'` like this:

```
'num_leaves': hp.choice('num_leaves', list(range(20, 250, 10))),
'subsample': hp.choice('subsample', [0.2, 0.4, 0.5, 0.6, 0.7, .8, .9]),
```

Do not use the above code! Rather, these should be defined as below so Hyperopt can efficiently explore the space:

```
'num_leaves': hp.quniform('num_leaves', 20, 250, 10),
'subsample': hp.uniform('subsample', 0.2, .9),
```

The `uniform` function can specify a uniform distribution over a continuous range of values. This function takes two arguments: the minimum and maximum values of the uniform distribution, and returns a random floating-point value within this range.

For example, to generate a random floating-point value between 0 and 1 using the uniform function, you can use the following code:

```
>>> from hyperopt import hp, pyll

>>> pyll.stochastic.sample(hp.uniform('value', 0, 1))
0.7875384438202859
```

Let's call it 10,000 times and then look at a histogram of the result. You can see that it has an equal probability of choosing any value.

```
uniform_vals = [pyll.stochastic.sample(hp.uniform('value', 0, 1))
                for _ in range(10_000)]

fig, ax = plt.subplots(figsize=(8, 4))
ax.hist(uniform_vals)
```

The `loguniform` function can specify a log-uniform distribution over a continuous range of values. This function takes two arguments: the minimum and maximum values of the log-uniform distribution, and it returns a random floating-point value within this range on a logarithmic scale.

Here is the histogram for the loguniform values from -5 to 5. Note that these values do not range from -5 to 5, but rather 0.006737 (`math.exp(-5)`) to 148.41 (`math.exp(5)`). Note that these values will strongly favor the low end.

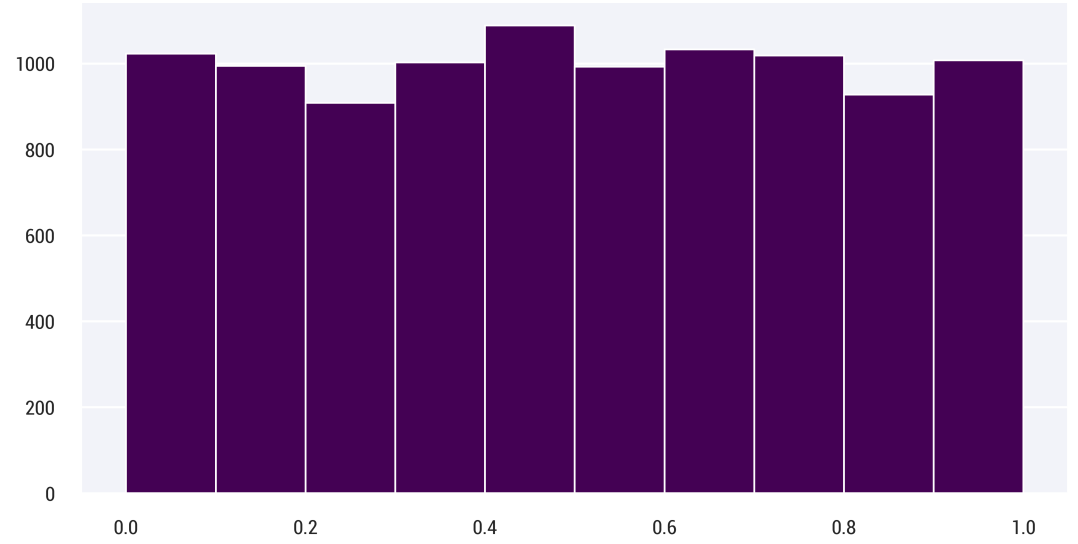


Figure 12.1: Plot of the histogram of uniform values for low=0 and high=1

```
loguniform_vals = [pyll.stochastic.sample(hp.loguniform('value', -5, 5))
                   for _ in range(10_000)]

fig, ax = plt.subplots(figsize=(8, 4))
ax.hist(loguniform_vals)
```

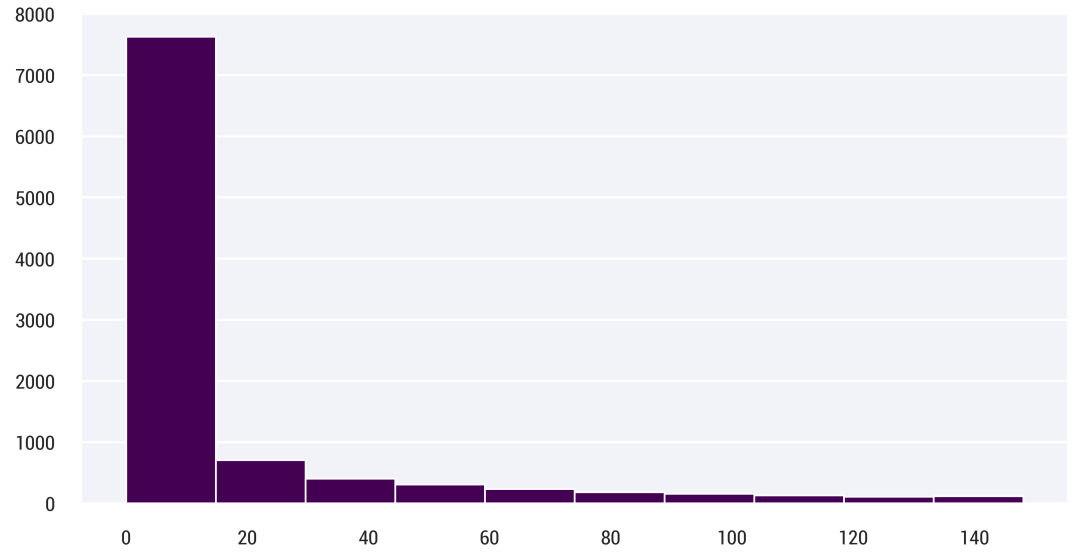


Figure 12.2: Plot of the histogram of loguniform values for low=-5 and high=5

Here is another way to look at it. This is a plot of the transform of possible values when the min and max values are set to -5 and 5, respectively. In the y-axis are the exponential values that come out of the `loguniform` function.

```
fig, ax = plt.subplots(figsize=(8, 4))
(pd.Series(np.arange(-5, 5, step=.1))
 .rename('x')
 .to_frame()
 .assign(y=lambda adf:np.exp(adf.x))
 .plot(x='x', y='y', ax=ax)
)
```

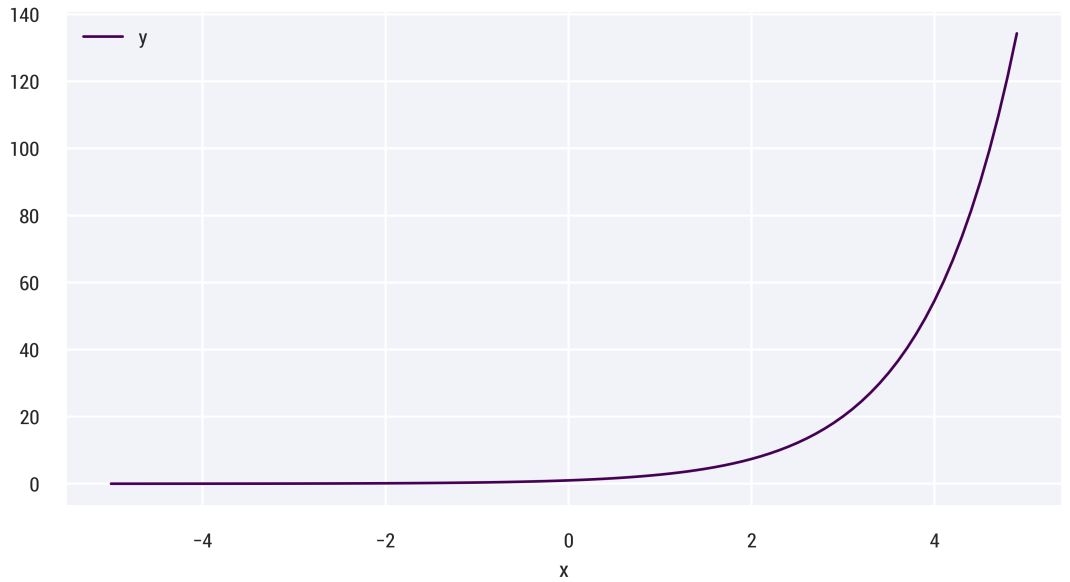


Figure 12.3: Plot of the loguniform values for low=-5 and high=5

For example, to generate a random floating-point value between 0.1 and 10 using the `loguniform` function, you can use the following code:

```
>>> from hyperopt import hp, pyll
>>> from math import log
>>> pyll.stochastic.sample(hp.loguniform('value', log(.1), log(10)))
3.0090767867889174
```

If you want values pulled from a log scale, .001, .01, .1, 1, 10, then use `loguniform`.

The `quniform` function can be used to specify an integer from the range [exp(low), exp(high)]. The function also accepts a `q` parameter that specifies the step (for `q=1`, it returns every integer in the range. For `q=3`, it returns every third integer).

```
quniform_vals = [pyll.stochastic.sample(hp.quniform('value', -5, 5, q=2))
                 for _ in range(10_000)]
```

Here are the counts for the `quniform` values from -5 to 5, taking every other value (`q=2`).

```
>>> pd.Series(quniform_vals).value_counts()
-0.0    2042
-2.0    2021
 2.0    2001
 4.0    2000
-4.0    1936
dtype: int64
```

12.4 Exploring the Trials

The `trial` object from the hyperopt search above has the data about the hyperparameter optimization.

Feel free to skip this section if you want, but I want to show you how to explore this and see what Hyperopt is doing.

First, I will make a function to convert the trial object to a Pandas dataframe.

```
from typing import Any, Dict, Sequence
def trial2df(trial: Sequence[Dict[str, Any]]) -> pd.DataFrame:
    """
    Convert a Trial object (sequence of trial dictionaries)
    to a Pandas DataFrame.

    Parameters
    ----------
    trial : List[Dict[str, Any]]
        A list of trial dictionaries.

    Returns
    -------
    pd.DataFrame
        A DataFrame with columns for the loss, trial id, and
        values from each trial dictionary.
    """
    vals = []
    for t in trial:
        result = t['result']
        misc = t['misc']
        val = {k:(v[0] if isinstance(v, list) else v)
               for k,v in misc['vals'].items()
              }
        val['loss'] = result['loss']
        val['tid'] = t['tid']
        vals.append(val)
    return pd.DataFrame(vals)
```

Now, let's use the function to create a dataframe.

```
>>> hyper2hr = trial2df(trials)
```

Each row is an evaluation. You can see the hyperparameter settings, the loss score, and the trial id (*tid*).

```
>>> hyper2hr
      colsample_bytree       gamma  learning_rate  ...  subsample      loss  \
0             0.854670    2.753933       0.042056  ...   0.913247 -0.744751
1             0.512653    0.153628       0.611973  ...   0.550048 -0.746961
2             0.552569    1.010561       0.002412  ...   0.508593 -0.735912
3             0.604020  682.836185       0.005037  ...   0.536935 -0.545856
4             0.785281    0.004130       0.015200  ...   0.691211 -0.739227
...                ...         ...            ...  ...        ...       ...
1995          0.717890    0.000543       0.141629  ...   0.893414 -0.765746
1996          0.725305    0.000248       0.172854  ...   0.919415 -0.765746
1997          0.698025    0.028484       0.162207  ...   0.952204 -0.770166
1998          0.688053    0.068223       0.099814  ...   0.939489 -0.762431
1999          0.666225    0.125253       0.203441  ...   0.980354 -0.767956

       tid
0        0
1        1
2        2
3        3
4        4
...    ...
1995  1995
1996  1996
1997  1997
1998  1998
1999  1999

[2000 rows x 10 columns]
```

I'll do some EDA (exploratory data analysis) on this data.

I like to look at correlations for numeric data and it might be interesting to see if hyperparameters are correlated to each other or to the loss. We can also inspect if a feature has a monotonic relationship with the loss score.

Here is a Seaborn plot to show the correlations.

```
import seaborn as sns
fig, ax = plt.subplots(figsize=(8, 4))
sns.heatmap(hyper2hr.corr(method='spearman'),
    cmap='RdBu', annot=True, fmt='.2f', vmin=-1, vmax=1, ax=ax
)
```

Note

If you want to create a correlation table in Jupyter with Pandas, you can use this code:

```
(hyper2hr
 .corr(method='spearman')
 .style
```

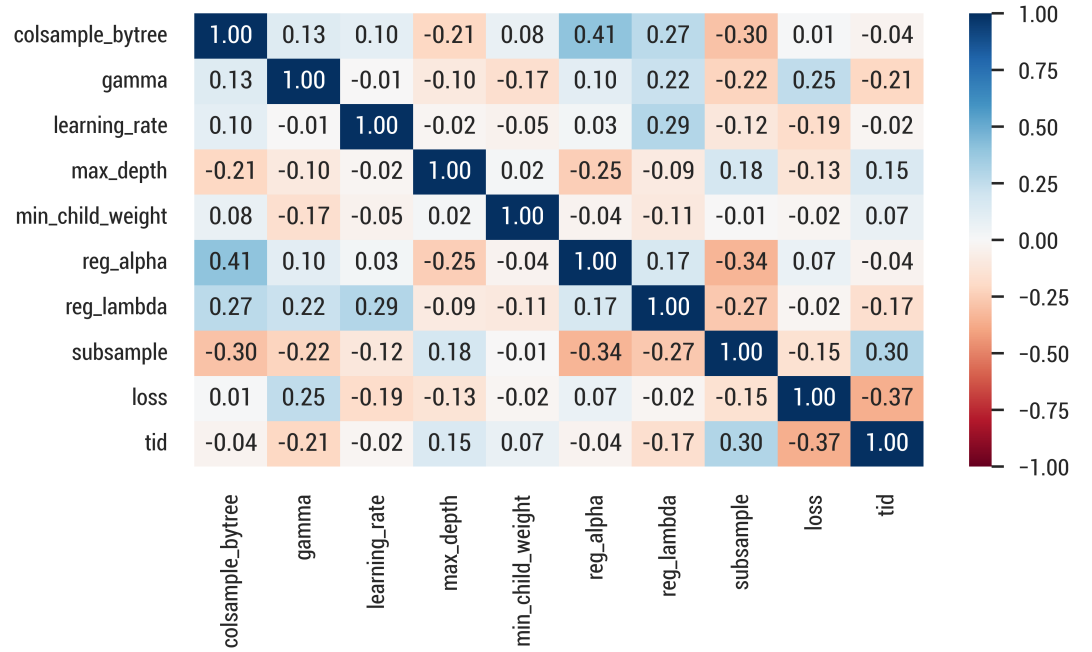


Figure 12.4: Spearman correlation of hyperparameters during Hyperopt process.

```
      .background_gradient(cmap='RdBu', vmin=-1, vmax=1)
    )
```

There is a negative correlation between loss and *tid* (trial id). This is to be expected. As we progress, the negative loss score should drop. The optimization process should balance exploration and exploitation to minimize the negative loss score. If we do a scatter plot of these two columns, we see this play out. Occasionally, an exploration attempt proves poor (the loss value around -0.55), and the Bayesian search process cuts back to better values.

```
fig, ax = plt.subplots(figsize=(8, 4))
(hyper2hr
  .plot.scatter(x='tid', y='loss', alpha=.1, color='purple', ax=ax)
)
```

Let's explore what happened with *max_depth* and *olss* which have a correlation of -0.13.

```
fig, ax = plt.subplots(figsize=(8, 4))
(hyper2hr
  .plot.scatter(x='max_depth', y='loss', alpha=1, color='purple', ax=ax)
)
```

This is a little hard to understand because the *max_depth* values plot on top of each other. I will use a technique called *jittering* to pull them apart and let us understand how many values there are. I will also adjust the alpha values of the plot so we can see where values overlap.

Here is my `jitter` function. It adds a random amount of noise around a data point. Let's apply it to the *max_depth* column.

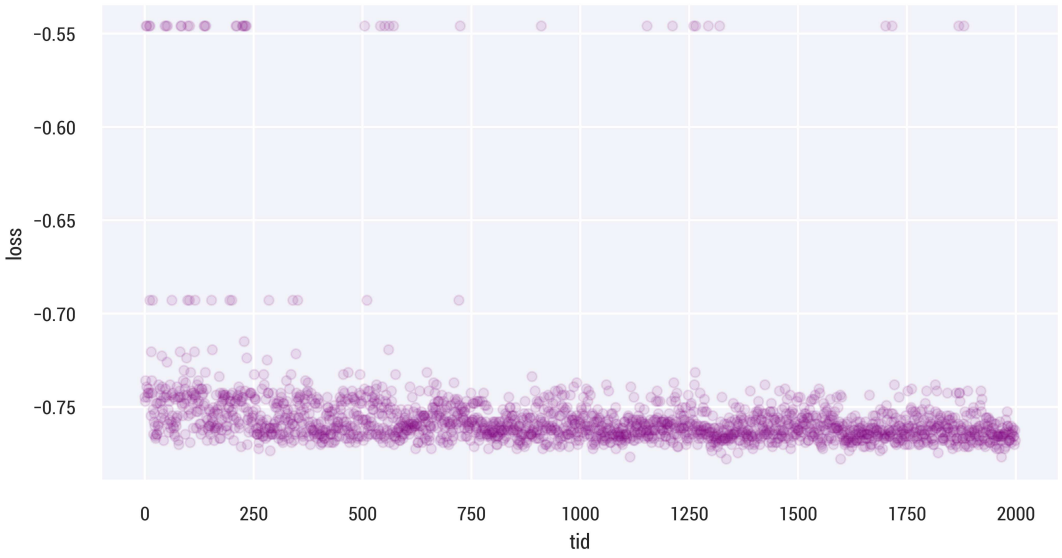


Figure 12.5: Scatter plot of loss vs trial during Hyperopt process. Note that the values tend to lower as the tid goes up.

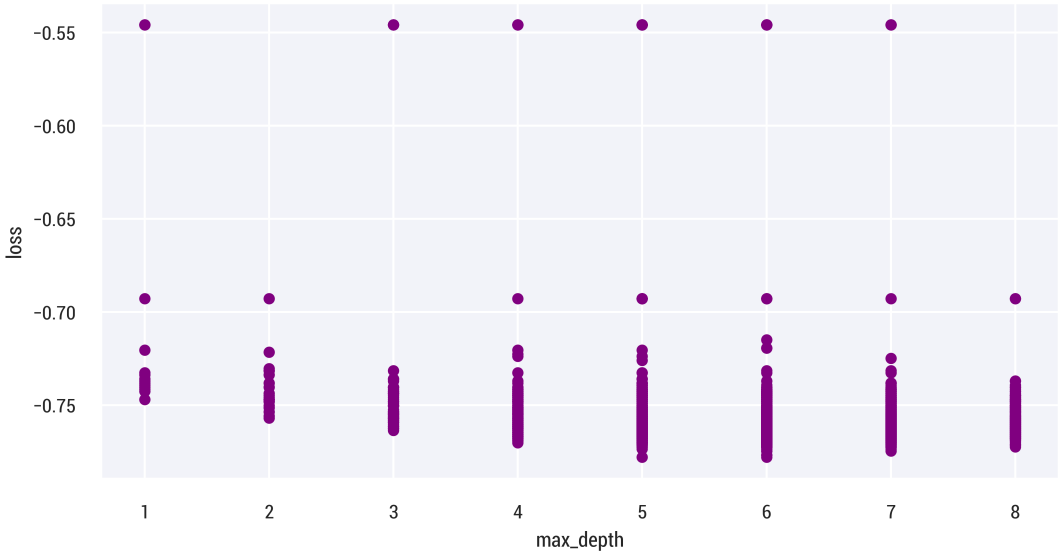


Figure 12.6: Scatter plot of loss vs max_depth during Hyperopt process.

```
import numpy as np

def jitter(df: pd.DataFrame, col: str, amount: float=1) -> pd.Series:
    """
    Add random noise to the values in a Pandas DataFrame column.

    This function adds random noise to the values in a specified
    column of a Pandas DataFrame. The noise is uniform random
    noise with a range of `amount` centered around zero. The
    function returns a Pandas Series with the jittered values.

    Parameters
    ----------
    df : pd.DataFrame
        The input DataFrame.
    col : str
        The name of the column to jitter.
    amount : float, optional
        The range of the noise to add. The default value is 1.

    Returns
    -------
    pd.Series
        A Pandas Series with the jittered values.
    """
    vals = np.random.uniform(low=-amount/2, high=amount/2,
                             size=df.shape[0])
    return df[col] + vals


fig, ax = plt.subplots(figsize=(8, 4))
(hyper2hr
  .assign(max_depth=lambda df:jitter(df, 'max_depth', amount=.8))
  .plot.scatter(x='max_depth', y='loss', alpha=.1, color='purple', ax=ax)
)
```

This makes it quite clear that the algorithm spent a good amount of time at depth 6.

If we want to get even fancier, we can color this by trial attempt. The later attempts are represented by the yellow color.

```
fig, ax = plt.subplots(figsize=(8, 4))
(hyper2hr
  .assign(max_depth=lambda df:jitter(df, 'max_depth', amount=.8))
  .plot.scatter(x='max_depth', y='loss', alpha=.5,
                color='tid', cmap='viridis', ax=ax)
)
```

We could also use Seaborn to create a violin plot of *loss* against *max_depth*. I don't feel like this lets me see the density as well.

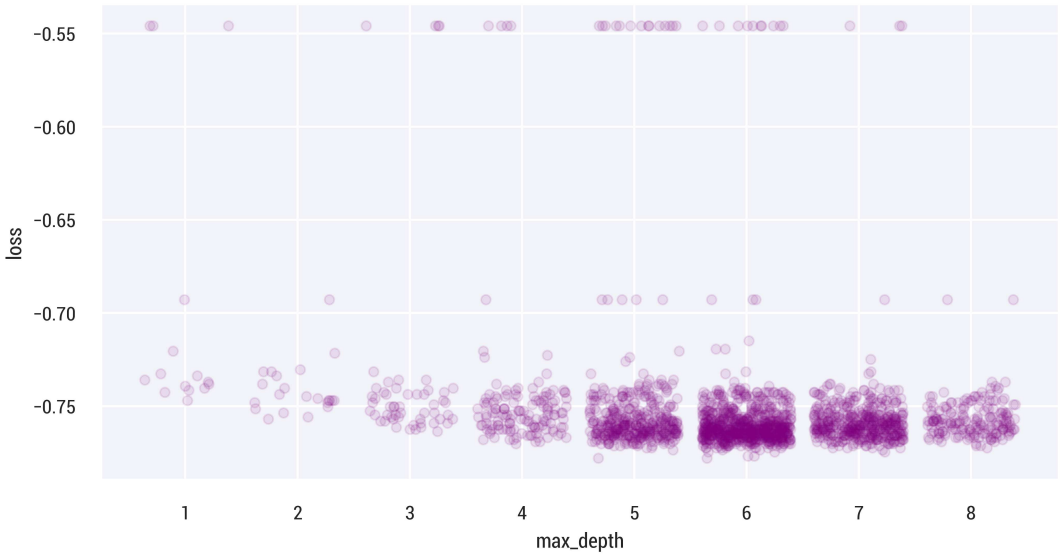


Figure 12.7: Scatter plot of loss vs max_depth during Hyperopt process with jittering. You can see the majority of the values were in the 5-7 range.

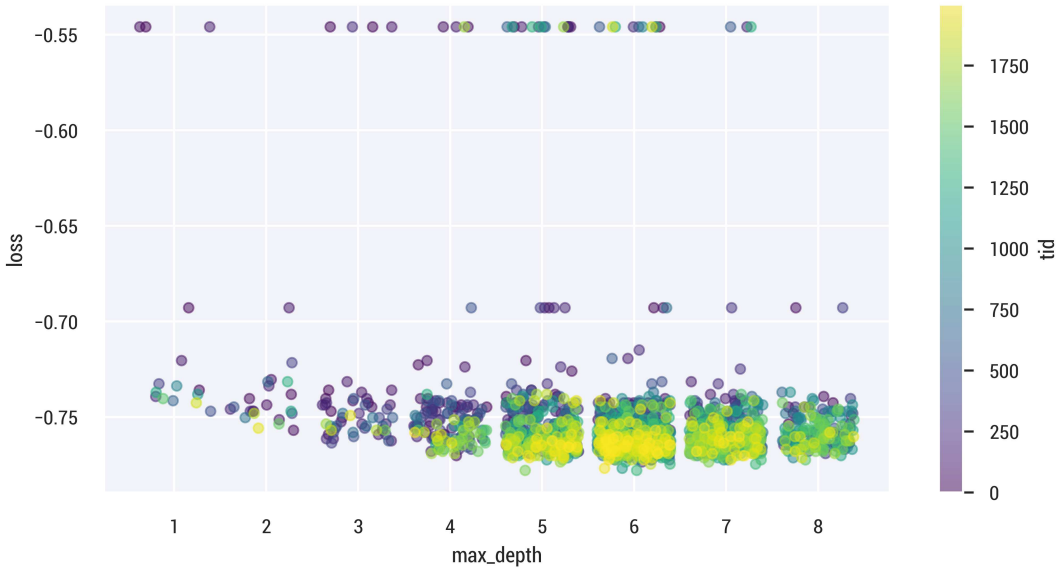


Figure 12.8: Scatter plot of loss vs max_depth during Hyperopt process colored by attempt.

```
import seaborn as sns
fig, ax = plt.subplots(figsize=(8, 4))
sns.violinplot(x='max_depth', y='loss', data=hyper2hr, kind='violin', ax=ax)
```

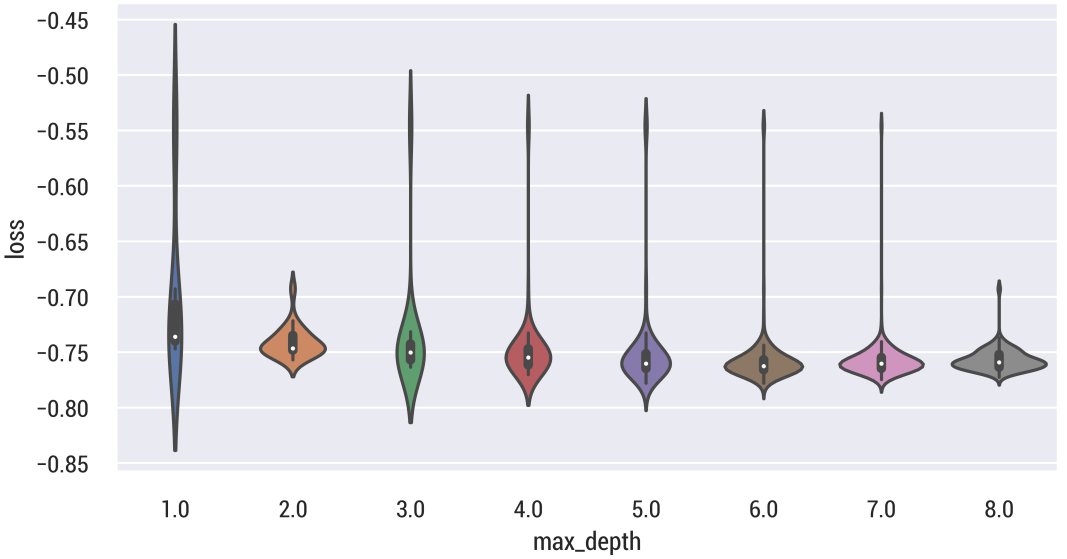


Figure 12.9: Violin plot of loss versus max_depth. I prefer the jittered scatter plot because it allows me to understand the density better.

The correlation between *reg_alpha* and *colsample_bytree* was 0.41, meaning there was a slight tendency for the rank of one value to go up if the rank of the other value went up. Let's plot this and see if we can glean any insight.

```
fig, ax = plt.subplots(figsize=(8, 4))
(hyper2hr
  .plot.scatter(x='reg_alpha', y='colsample_bytree', alpha=.8,
                color='tid', cmap='viridis', ax=ax)
)

ax.annotate('Min Loss (-0.77)', xy=(4.56, 0.692),
            xytext=(.7, .84), arrowprops={'color':'k'})
```

The values for *colsample_bytree* and *reg_alpha* were .68 and 3.29 respectively.
Let's explore *gamma* and *loss* that had a correlation of .25.
Here is my initial plot.

```
fig, ax = plt.subplots(figsize=(8, 4))
(hyper2hr
  .plot.scatter(x='gamma', y='loss', alpha=.1, color='purple', ax=ax)
)
```

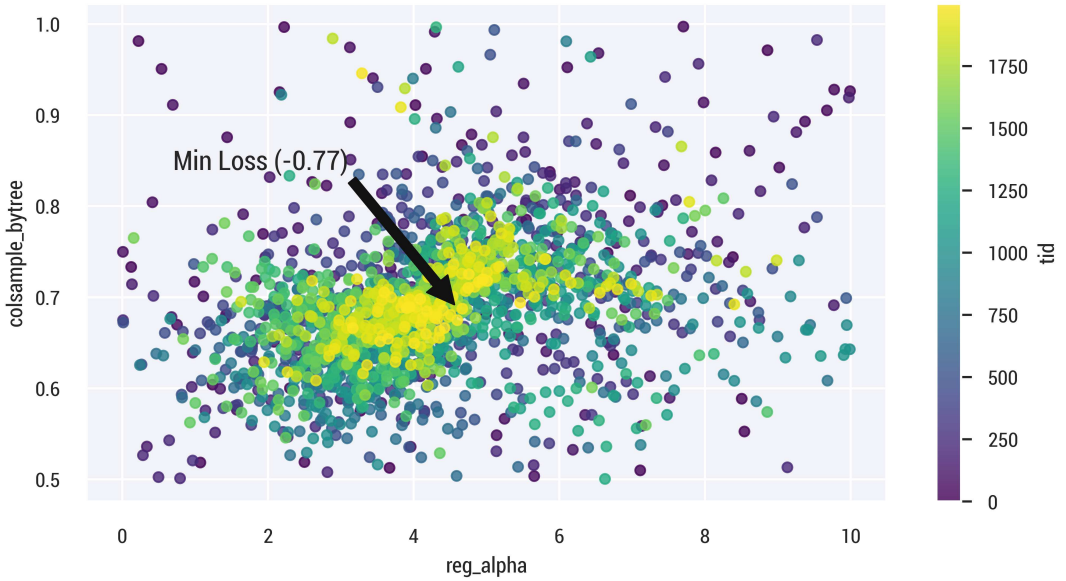


Figure 12.10: Scatter plot of colsample_bytree vs reg_alpha during Hyperopt process colored by attempt.

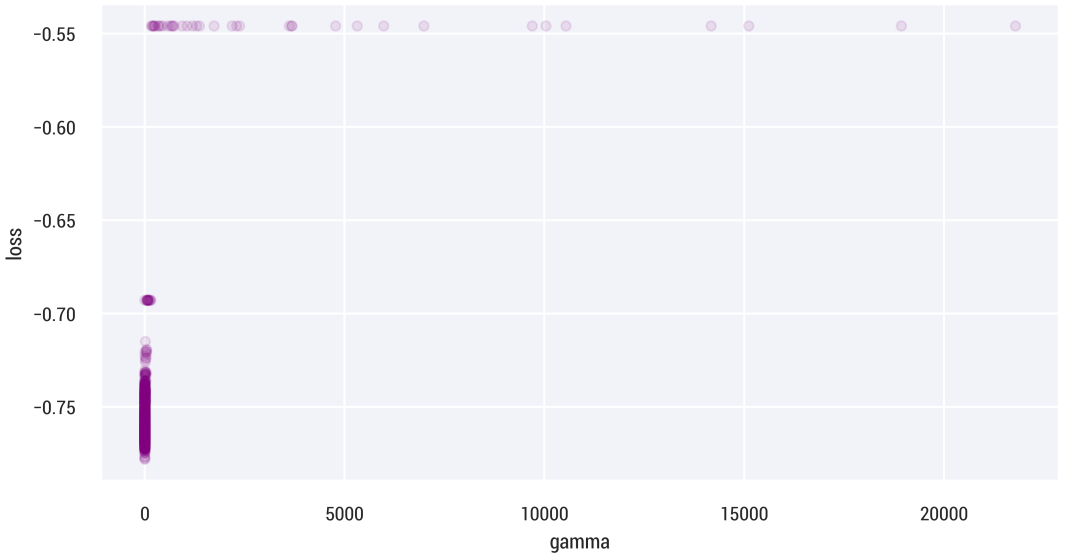


Figure 12.11: Scatter plot of loss vs gamma during Hyperopt process.

It is a little hard to tell what is going on here. The stacking on the left side might indicate a need for jittering. But if you look at the scale, you can see that there are some outliers.

Remember that *gamma* was defined with this distribution:

```
'gamma': hp.loguniform('gamma', -10, 10), # regularization
```

These values range from `math.exp(-10)` to `math.exp(10)`. Because this is a log-uniform distribution, they will tend towards the low end of this.

One trick you can use is to plot with a log x-axis. That will help tease these values apart.

```
fig, ax = plt.subplots(figsize=(8, 4))
(hyper2hr
  .plot.scatter(x='gamma', y='loss', alpha=.5, color='tid', ax=ax,
                logx=True, cmap='viridis')
)

ax.annotate('Min Loss (-0.77)', xy=(0.000581, -0.777),
            xytext=(1, -.6), arrowprops={'color':'k'})
```

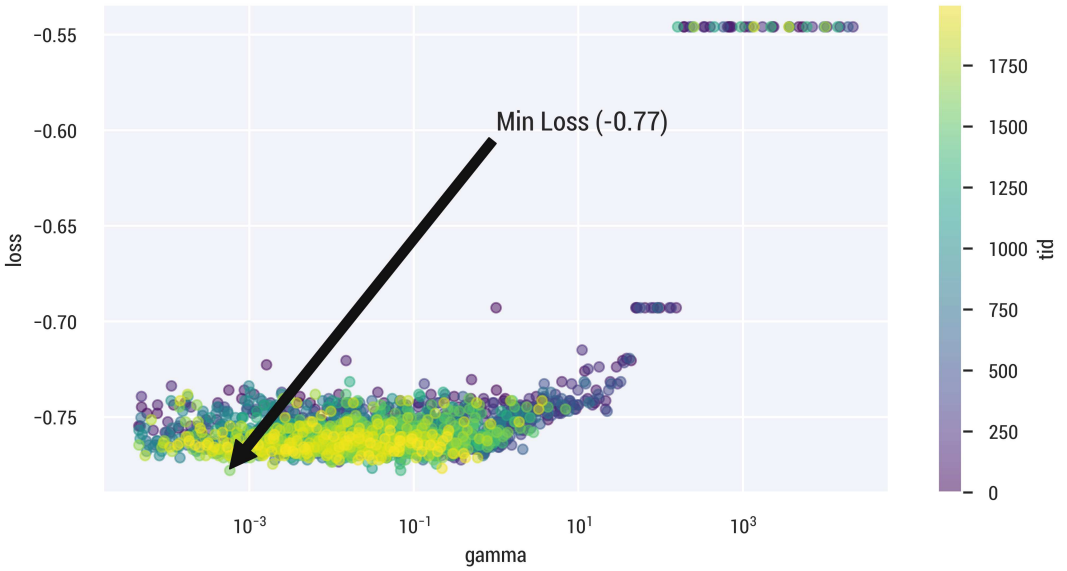


Figure 12.12: Scatter plot of loss vs gamma during Hyperopt process with log x-axis

This shows how Bayesian searching focuses on the values that provide the best loss. Occasionally, it does some exploring, but any gamma value above 10 performs poorly. The gamma value that it settled on during this run was 0.000581000581.

12.5 EDA with Plotly

Feel free to skip this section too. I like digging into the data. I find the Plotly library works well if I was to create 3D plots and have some interactivity.

Here is a helper function I wrote that will create a 3D mesh. I will make a mesh of two hyperparameters and plot the score in the third (Z) dimension.

```
import plotly.graph_objects as go

def plot_3d_mesh(df: pd.DataFrame, x_col: str, y_col: str,
                 z_col: str) -> go.Figure:
    """
    Create a 3D mesh plot using Plotly.

    This function creates a 3D mesh plot using Plotly, with
    the `x_col`, `y_col`, and `z_col` columns of the `df`
    DataFrame as the x, y, and z values, respectively. The
    plot has a title and axis labels that match the column
    names, and the intensity of the mesh is proportional
    to the values in the `z_col` column. The function returns
    a Plotly Figure object that can be displayed or saved as
    desired.

    Parameters
    ----------
    df : pd.DataFrame
        The DataFrame containing the data to plot.
    x_col : str
        The name of the column to use as the x values.
    y_col : str
        The name of the column to use as the y values.
    z_col : str
        The name of the column to use as the z values.

    Returns
    -------
    go.Figure
        A Plotly Figure object with the 3D mesh plot.
    """
    fig = go.Figure(data=[go.Mesh3d(x=df[x_col], y=df[y_col], z=df[z_col],
        intensity=df[z_col]/ df[z_col].min(),
        hovertemplate=f"{z_col}: %{{z}}<br>{x_col}: %{{x}}<br>{y_col}: "
                                    "%{{y}}<extra></extra>")],
    )

    fig.update_layout(
        title=dict(text=f'{y_col} vs {x_col}'),
        scene = dict(
          xaxis_title=x_col,
          yaxis_title=y_col,
          zaxis_title=z_col),
        width=700,
        margin=dict(r=20, b=10, l=10, t=50)
    )
    return fig
```

Here is the code to plot a mesh of *gamma* and *reg_lambda*. In the z-axis, I plot the *loss*. The minimum value colors this in the z-axis. The most yellow value has the least loss.

Plotly allows us to interact with the plot, rotate it, zoom in or out, and show values by hovering.

```
fig = plot_3d_mesh(hyper2hr.query('gamma < .2'),
    'reg_lambda', 'gamma', 'loss')

fig
```

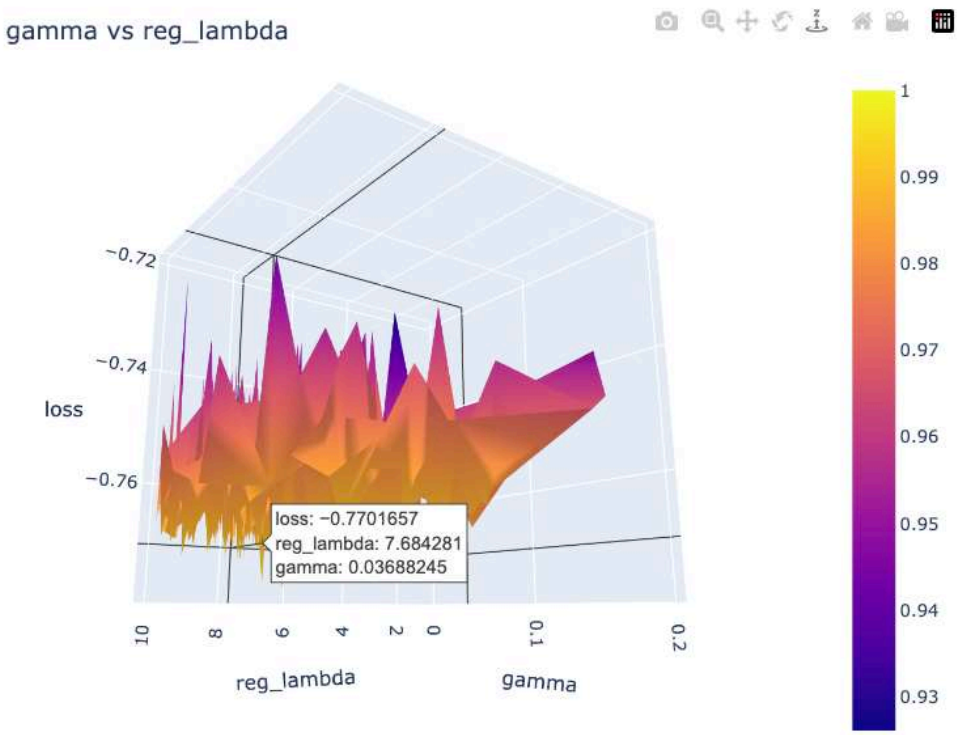


Figure 12.13: Scatter plot of loss vs gamma during Hyperopt process with log x-axis

Here is the code to create a scatter plot. (It turns out that this wrapper isn't really needed but I provided it for consistency).

```
import plotly.express as px
import plotly.graph_objects as go

def plot_3d_scatter(df: pd.DataFrame, x_col: str, y_col: str,
                z_col: str, color_col: str,
                opacity: float=1) -> go.Figure:
    """
    Create a 3D scatter plot using Plotly Express.

    This function creates a 3D scatter plot using Plotly Express,
    with the `x_col`, `y_col`, and `z_col` columns of the `df`
    DataFrame as the x, y, and z values, respectively. The points
```

```
    in the plot are colored according to the values in the
    `color_col` column, using a continuous color scale. The
    function returns a Plotly Express scatter_3d object that
    can be displayed or saved as desired.

    Parameters
    ----------
    df : pd.DataFrame
        The DataFrame containing the data to plot.
    x_col : str
        The name of the column to use as the x values.
    y_col : str
        The name of the column to use as the y values.
    z_col : str
        The name of the column to use as the z values.
    color_col : str
        The name of the column to use for coloring.
    opacity : float
        The opacity (alpha) of the points.

    Returns
    -------
    go.Figure
        A Plotly Figure object with the 3D mesh plot.
    """
    fig = px.scatter_3d(data_frame=df, x=x_col,
                y=y_col, z=z_col, color=color_col,
                color_continuous_scale=px.colors.sequential.Viridis_r,
                opacity=opacity)
    return fig
```

Let's look at *gamma* and *reg_lambda* over time. I'm using color for *loss*, so the high values in the z-axis should be getting better loss scores as the hyperparameter search progresses.

```
plot_3d_scatter(hyper2hr.query('gamma < .2'),
                'reg_lambda', 'gamma', 'tid', color_col='loss')
```

12.6 Conclusion

In this chapter, we introduced the Hyperopt library and performed hyperparameter searches using Bayesian optimization. Since there are so many hyperparameters for XGBoost, using grid search to optimize them could take a long time because it will explore values known to be bad. Hyperopt can focus on and around known good values.

We also showed how to specify parameter distributions and gave examples of the different types of distributions. We showed good default distributions that you can use for your models.

Finally, we explored the relationships between hyperparameters and loss scores using pandas and visualization techniques.

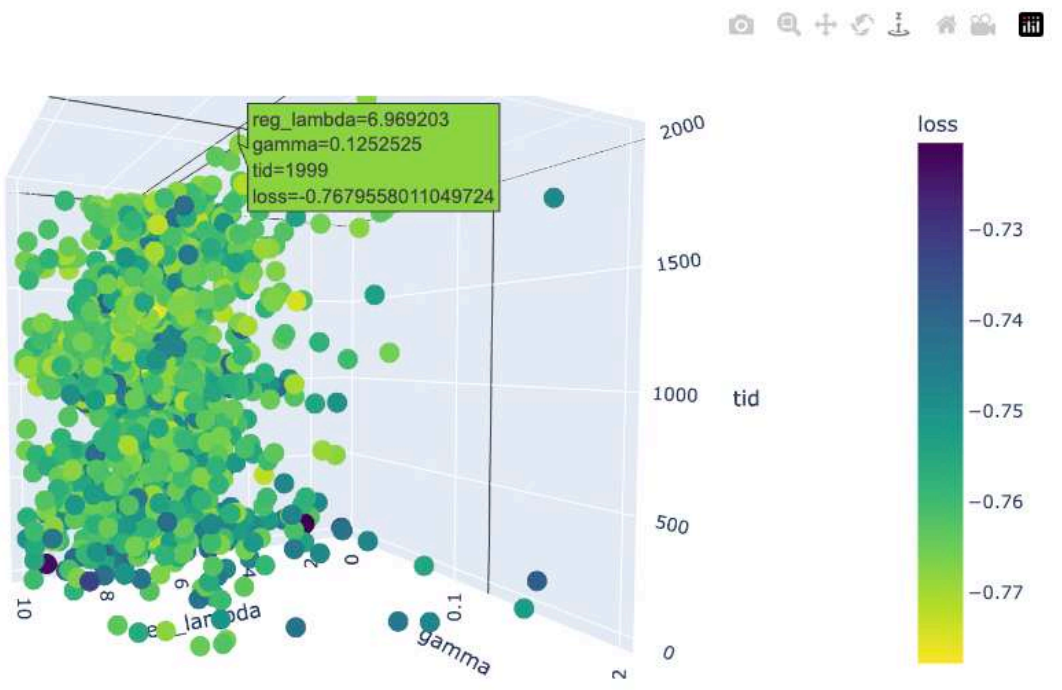


Figure 12.14: Scatter plot of loss vs gamma during Hyperopt process with log x-axis

12.7 Exercises

1. What is the Hyperopt library and how does it differ from grid search for optimizing hyperparameters?
2. How can parameter distributions be specified in Hyperopt? Give an example of a continuous distribution and a discrete distribution.
3. What are some good default distributions for your models in Hyperopt?
4. How can pandas and visualization techniques be used to explore the relationships between hyperparameters and loss scores?

Chapter 13

Step-wise Tuning with Hyperopt

In this chapter, we introduce *step-wise tuning* as a method for optimizing the hyperparameters of an XGBoost model. The main reason for tuning with this method is to save time. This method involves tuning small groups of hyperparameters that act similarly and then moving on to the next group while preserving the values from the previous group. This can significantly reduce the search space compared to tuning all hyperparameters simultaneously.

13.1 Groups of Hyperparameters

In this section, rather than tuning all of the parameters at once, we will use *step-wise tuning*. We will tune small groups of hyperparameters that act similarly, then move to the next group while preserving the values from the previous group. I'm going to limit my steps to:

- Tree parameters
- Sampling parameters
- Regularization parameters
- Learning rate

This limits the search space by quite a bit. Rather than searching 100 options for trees for every 100 options for sampling (10,000), another 100 options for regularization (1,000,000), and another 100 for learning rate (100,000,000), you are searching 400 options! Of course, it could be that this finds a local maximum and ignores the interplay between hyperparameters, but I find that it is often a worthwhile tradeoff if you are pressed for time.

Here is the code that I use for step-wise tuning. In the `rounds` list, I have a dictionary for the hyperparameters to evaluate for each step. The `max_evals` setting in the call to `fmin` determines how many attempts hyperopt makes during the round. You can bump up this number if you want it to explore more values.

```
from hyperopt import fmin, tpe, hp, Trials
params = {'random_state': 42}

rounds = [{'max_depth': hp.quniform('max_depth', 1, 8, 1),  # tree
           'min_child_weight': hp.loguniform('min_child_weight', -2, 3)},
          {'subsample': hp.uniform('subsample', 0.5, 1),   # stochastic
           'colsample_bytree': hp.uniform('colsample_bytree', 0.5, 1)},
          {'reg_alpha': hp.uniform('reg_alpha', 0, 10),
            'reg_lambda': hp.uniform('reg_lambda', 1, 10),},
          {'gamma': hp.loguniform('gamma', -10, 10)}, # regularization
          {'learning_rate': hp.loguniform('learning_rate', -7, 0)} # boosting
```

```
]

all_trials = []
for round in rounds:
    params = {**params, **round}
    trials = Trials()
    best = fmin(fn=lambda space: xhelp.hyperparameter_tuning(space, X_train,
                                        y_train, X_test, y_test),
        space=params,
        algo=tpe.suggest,
        max_evals=20,
        trials=trials,
    )
    params = {**params, **best}
    all_trials.append(trials)
```

Rather than taking 30 minutes (or two hours om my slower computer) to exhaustively train all of the options, this finished in a little over a minute.

13.2 Visualization Hyperparameter Scores

Let's use the Plotly visualization code from the last chapter to explore the relationship between *reg_alpha* and *reg_lambda*.

```
xhelp.plot_3d_mesh(xhelp.trial2df(all_trials[2]),
    'reg_alpha', 'reg_lambda', 'loss')
```

This is a lot coarser than our previous plots. You might use this to diagnose if you need to up the number of `max_evals`.

13.3 Training an Optimized Model

With the optimized parameters in hand, let's train a model with them. Make sure that you explicitly print out the values (this also helps from having to run through the search space again if you need to restart the notebook).

```
step_params = {'random_state': 42,
 'max_depth': 5,
 'min_child_weight': 0.6411044640540848,
 'subsample': 0.9492383155577023,
 'colsample_bytree': 0.6235721099295888,
 'gamma': 0.00011273797329538491,
 'learning_rate': 0.24399020050740935}
```

Once the parameters are in a dictionary, you can use dictionary unpacking (the ** operator) to pass the parameters into the constructor. Remember to set `early_stopping_rounds`, `n_estimators`, and provide an `eval_set` to the `.fit` method so that the number of trees is optimized (if you use all of the trees, bump up the `n_estimators` number and run again):

```
xg_step = xgb.XGBClassifier(**step_params, early_stopping_rounds=50,
                            n_estimators=500)
xg_step.fit(X_train, y_train,
```

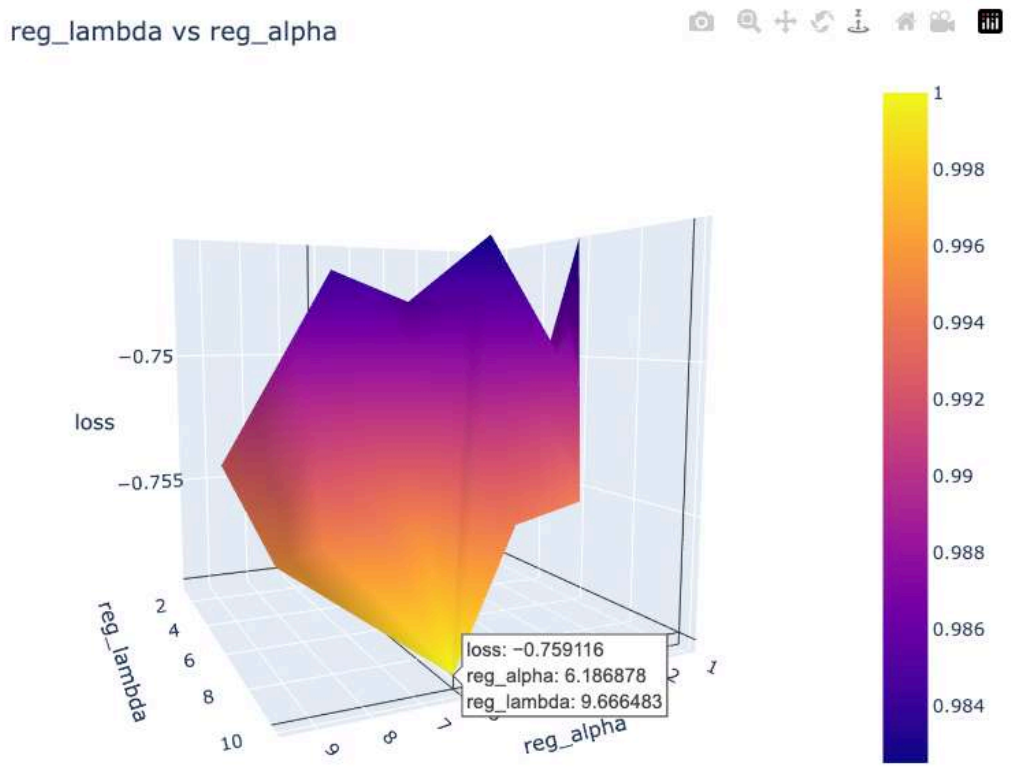


Figure 13.1: Contour plot of lambda and alpha during Hyperopt process

```
    eval_set=[(X_train, y_train),
              (X_test, y_test)
             ],
     verbose=100
    )
```

How does this model perform?

```
>>> xg_step.score(X_test, y_test)
0.7613259668508288
```

Looks pretty good! Let's compare this to the default out-of-the-box model.

```
>>> xg_def = xgb.XGBClassifier()
>>> xg_def.fit(X_train, y_train)
>>> xg_def.score(X_test, y_test)
0.7458563535911602
```

Our tuned model performs marginally better. However, a marginal improvement often yields outsized results.

13.4 Summary

In this chapter, we used the hyperopt library to optimize the performance of our XGBoost model. Hyperopt is a library for hyperparameter optimization that provides a range of search algorithms, parameter distributions, and performance metrics that can be used to optimize the hyperparameters of XGBoost models.

We used step-wise optimization to speed up the search. This may return a local minimum, but it does a decent job if you need an answer quickly. If you can access a cluster and have a weekend to burn, kick off a non-step-wise search.

13.5 Exercises

1. How does step-wise tuning with Hyperopt differ from traditional hyperparameter tuning methods such as grid search?
2. When does step-wise tuning make sense? When would you use a different strategy?

Chapter 14

Do you have enough data?

It is essential to have enough data for machine learning because the performance of a machine learning model is highly dependent on the amount and quality of the data used to train the model. A model trained on a large and diverse dataset will likely have better accuracy, generalization, and robustness than a model trained on a small and homogeneous dataset.

Having enough data for machine learning can help to overcome overfitting. Overfitting occurs when the model learns the noise and irrelevant details in the training data, leading to poor generalization when predicting with new data.

This chapter will introduce learning curves, a valuable tool for understanding your model.

14.1 Learning Curves

A *learning curve* is a graphical representation of the relationship between the performance of a machine-learning model and the amount of data used to train the model. We train a model on increasing amounts of data and plot the scores along the way. The x-axis of a learning curve shows the number of training examples or the amount of training data, and the y-axis shows the performance metric of the model, such as the error rate, accuracy, or F1 score.

The Yellowbrick library contains a learning curve visualization function. Note that I'm manually setting the y-limit for all the learning curve plots to help consistently convey the results. Otherwise, the plot would lop off the bottom portions essentially cropping the image and making better-performing models look as if they performed worse.

```
params = {'learning_rate': 0.3,
 'max_depth': 2,
 'n_estimators': 200,
 'n_jobs': -1,
 'random_state': 42,
 'reg_lambda': 0,
 'subsample': 1}

import yellowbrick.model_selection as ms
fig, ax = plt.subplots(figsize=(8, 4))
viz = ms.learning_curve(xgb.XGBClassifier(**params),
      X, y, ax=ax
)
ax.set_ylim(0.6, 1)
```

The learning curve for a good model illustrates a few things.

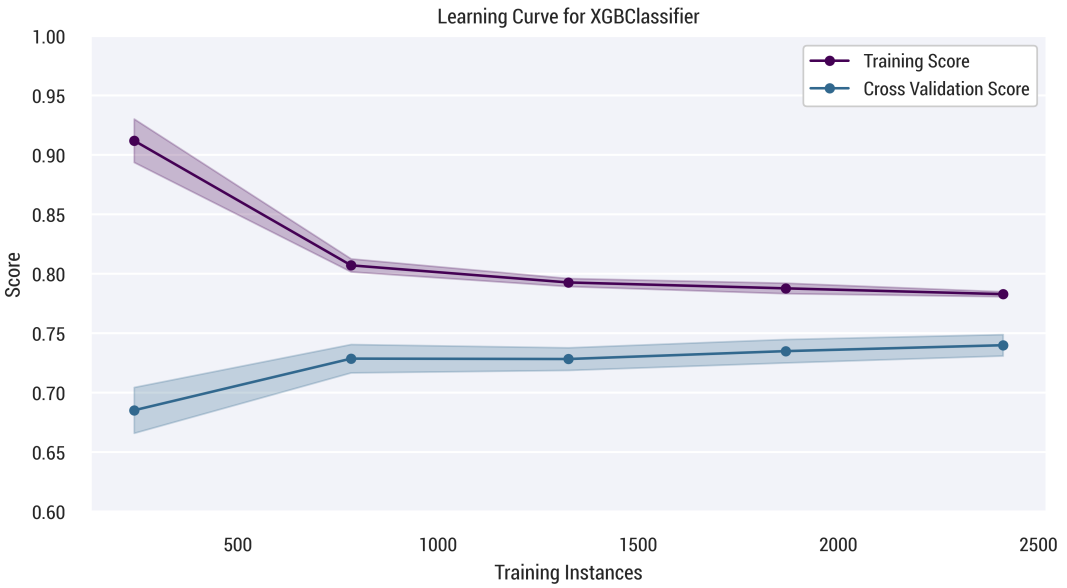


Figure 14.1: Learning curve for xgboost model

1. Shows a consistent and monotonic improvement in the model's cross-validation (or testing) performance as the amount of training data increases. This means that the error rate or accuracy of the model will decrease as the number of training examples or the amount of training data increases. It seems we could have an even better model if we added more data because the cross-validation score does not look like it has plateaued.
2. Learns the underlying patterns and trends in the data and that it is not overfitting or underfitting to the training data. An overfit model would show the training score trending close to 100% accuracy. We want to see it coming down to the testing data. We also like to see the cross-validation score trending close to the training score.

My takeaway from looking at this plot is that the model seems to be performing ok, it is not overfitting, and it might do even better if we had more data.

14.2 Learning Curves for Decision Trees

In the previous learning curve, we saw the behavior of an XGBoost model.

Let's plot a decision tree with a max depth of 7. This depth was a good compromise between overfitting and underfitting our model.

```
# tuned tree
fig, ax = plt.subplots(figsize=(8, 4))
viz = ms.learning_curve(tree.DecisionTreeClassifier(max_depth=7),
      X, y, ax=ax)
viz.ax.set_ylim(0.6, 1)
```

It looks similar to the XGBoost model in shape, but the scores are shifted down a bit.

My takeaway is similar to the XGBoost model. It looks pretty good. We also might want to get some more data.

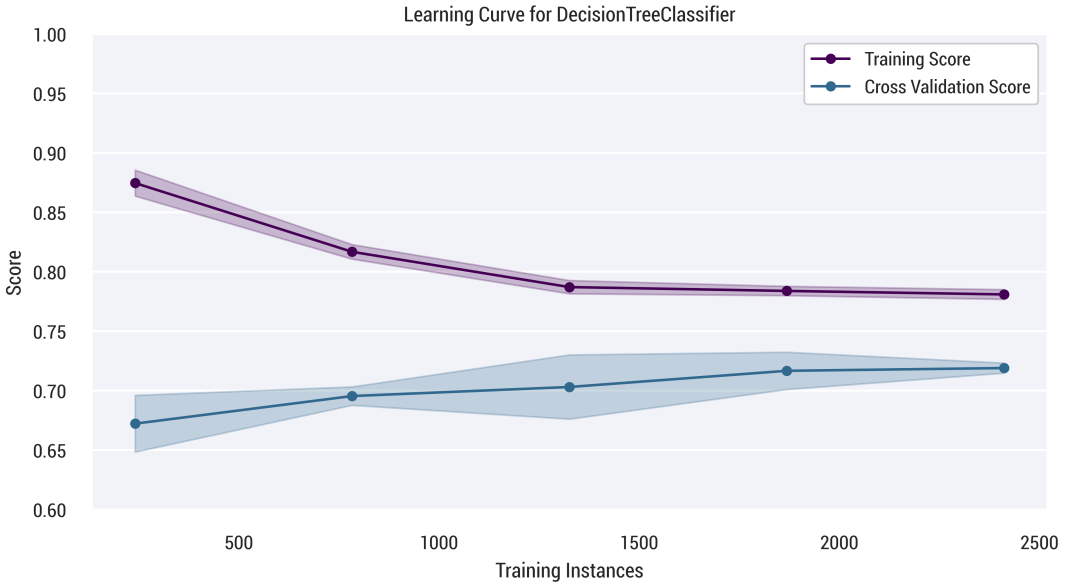


Figure 14.2: Learning curve for decision tree with depth of 7

14.3 Underfit Learning Curves

Now let's look at an underfit model. A learning curve for a decision stump model will show a low and constant cross-validation score as the amount of training data increases. As the number of training examples increases, it shouldn't impact the model because the model is too simple to learn from the data. We would also see poor performance on the training data. The score for both the training and testing data should be similarly bad.

If you look at the learning curve, you see that our model cannot learn the underlying patterns in the data. It needs to be more complex.

```
# underfit
fig, ax = plt.subplots(figsize=(8, 4))
viz = ms.learning_curve(tree.DecisionTreeClassifier(max_depth=1),
      X, y, ax=ax
)
ax.set_ylim(0.6, 1)
```

14.4 Overfit Learning Curves

Let's jump to the other end of the spectrum, an overfit learning curve. We can overfit a decision tree by using the default parameters which let the tree grow unbounded. Here, we expect to see good performance on the training data but poor performance on the testing data. The model memorizes the training data but cannot generalize the information to new data.

This learning curve is an excellent example of a model that is too complex.

```
# overfit
fig, ax = plt.subplots(figsize=(8, 4))
```

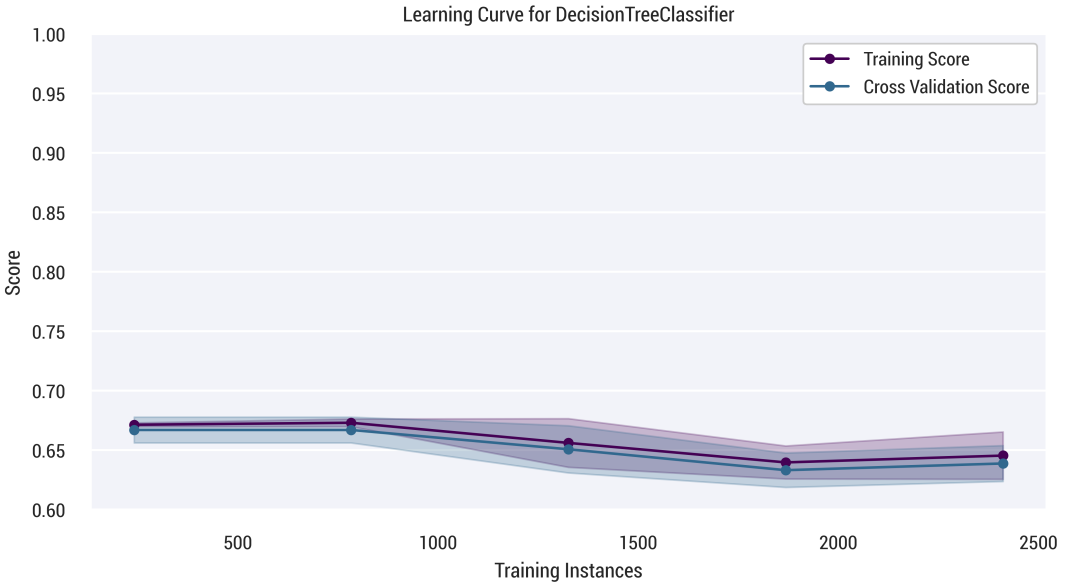


Figure 14.3: Learning curve for underfit decision stump

```
viz = ms.learning_curve(tree.DecisionTreeClassifier(),
      X, y, ax=ax
)
ax.set_ylim(0.6, 1)
```

14.5 Summary

Learning curves are a valuable tool to help diagnose models. We can infer whether adding more data will help the model perform better. We can also understand if the model is overfitting or underfitting.

If you want to know your machine learning model's health, plot a learning curve. Your model is on a good path if the learning curve is good. If the learning curve is bad, your model needs some attention, and you should consider adjusting its hyperparameters, adding more training data, or improving its features.

14.6 Exercises

1. What is the relationship between the performance of a machine learning model and the amount of data used to train the model?
2. How does having enough data for machine learning help to overcome overfitting?
3. What is a learning curve, and how is it used to evaluate the performance of a machine learning model?
4. What is the purpose of plotting a learning curve for a machine learning model?
5. What are some characteristics of a good learning curve for a machine learning model?
6. How can the learning curve of a model be used to determine if the model is overfitting or underfitting?

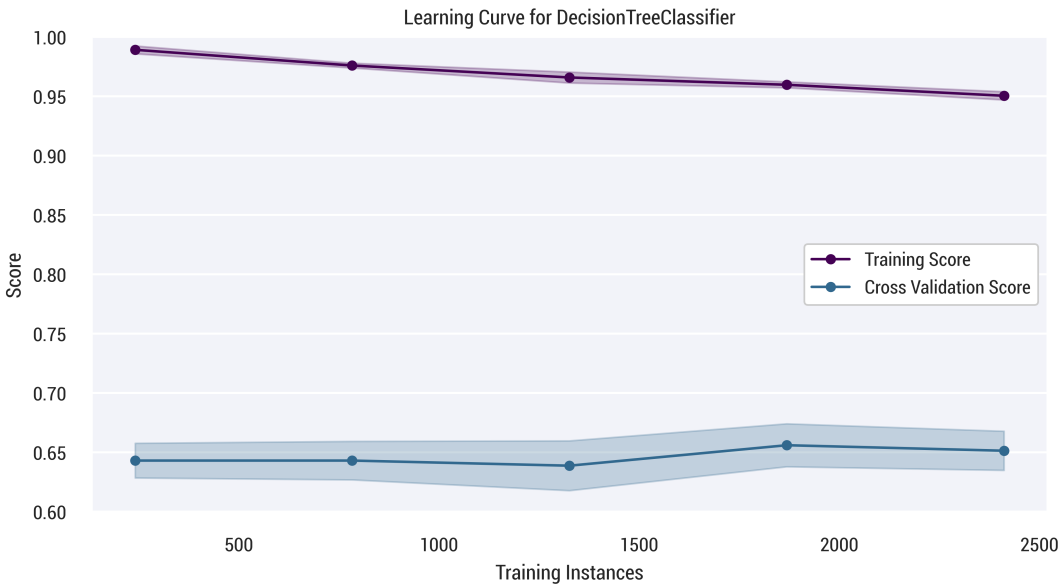


Figure 14.4: Learning curve for overfit decision tree. Evidence of overfitting is clear when the training score is high but the cross validation score never improves.

Chapter 15

Model Evaluation

Several machine learning evaluation techniques can be used to measure a machine learning model's performance, generalization, and robustness. In this chapter, we will explore many of them.

15.1 Accuracy

Let's start off with an out-of-the-box model and train it.

```
xgb_def = xgb.XGBClassifier()
xgb_def.fit(X_train, y_train)
```

The default result of calling the `.score` method returns the *accuracy*. The accuracy metric calculates the proportion of correct predictions made by the model. The metric is defined as the number of correct predictions divided by the total number of predictions.

```
>>> xgb_def.score(X_test, y_test)
0.7458563535911602
```

This model predicts 74% of the labels for the testing data correctly. On the face of it, it seems a valuable metric. But you must tread carefully with it, especially if you have imbalanced labels.

Imagine that you are building a machine learning model to predict whether a patient has cancer. You have a dataset of 100 patients, where 95 don't have cancer, and only 5 have cancer. You train a model that always predicts that a patient doesn't have cancer. This model will have an accuracy of 95%, which looks impressive but useless and possibly dangerous.

You can also use the `accuracy_score` function:

```
>>> from sklearn import metrics
>>> metrics.accuracy_score(y_test, xgb_def.predict(X_test))
0.7458563535911602
```

15.2 Confusion Matrix

A confusion matrix is a table that shows the number of true positive, true negative, false positive, and false negative predictions made by a machine learning model. The confusion matrix is useful because it helps you to understand the mistakes and errors of your machine learning model.

The confusion matrix aids in calculating prevalence, accuracy, precision, and recall.

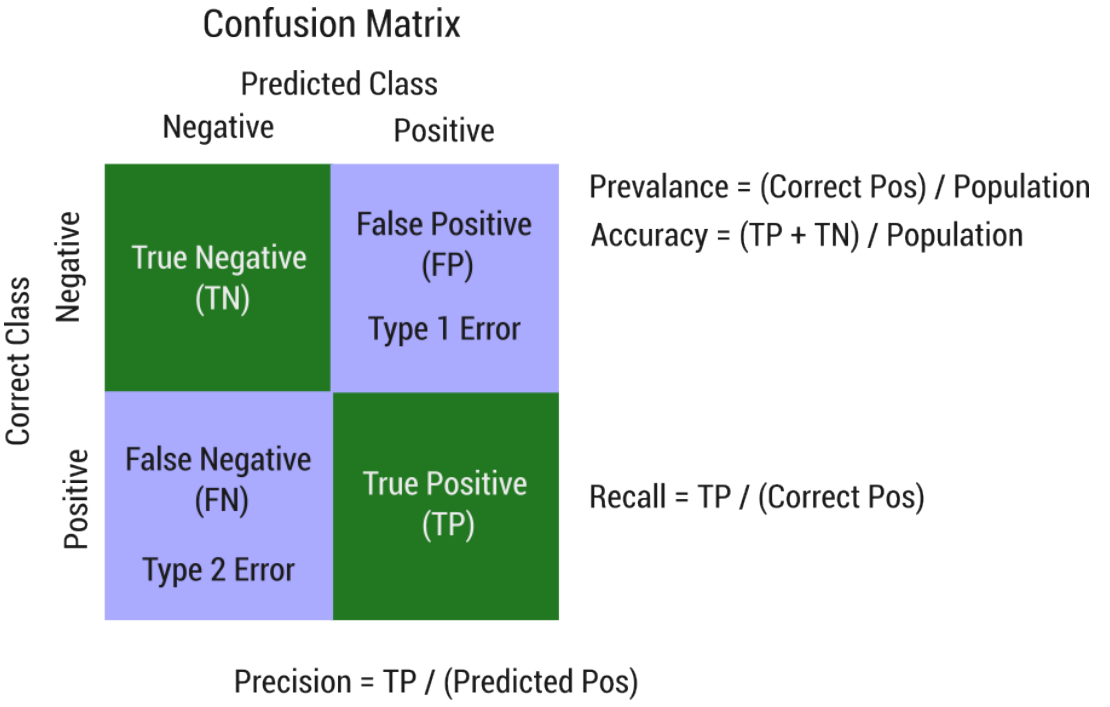


Figure 15.1: A confusion matrix and associated formulas.

Generally, the lower right corner represents the *true positive* values. Values that were predicted to have the positive label and were indeed positive. You want this number to be high. Above those of the *false positives* values. This is the count of values that were predicted to have the positive label but, in truth, were negative. You want this number to be low. (These are also called *type 1* errors by statisticians.)

In the upper left corner is the *true negative* values. Values that were both predicted and, in truth, negative. You want this value to be high. Below that value is the *false negative* count (or *type 2 errors*). These are positive values that were predicted as negative values.

```
fig, ax = plt.subplots(figsize=(8, 4))
classifier.confusion_matrix(xgb_def, X_train, y_train,
                            X_test, y_test,
                            classes=['DS', 'SE'], ax=ax
                           )
```

You can also use scikit-learn to create a NumPy matrix of a confusion matrix.

```
>>> from sklearn import metrics
>>> cm = metrics.confusion_matrix(y_test, xgb_def.predict(X_test))
>>> cm
array([[372, 122],
       [108, 303]])
```

Scikit-learn can create a matplotlib plot that is similar to Yellowbrick.

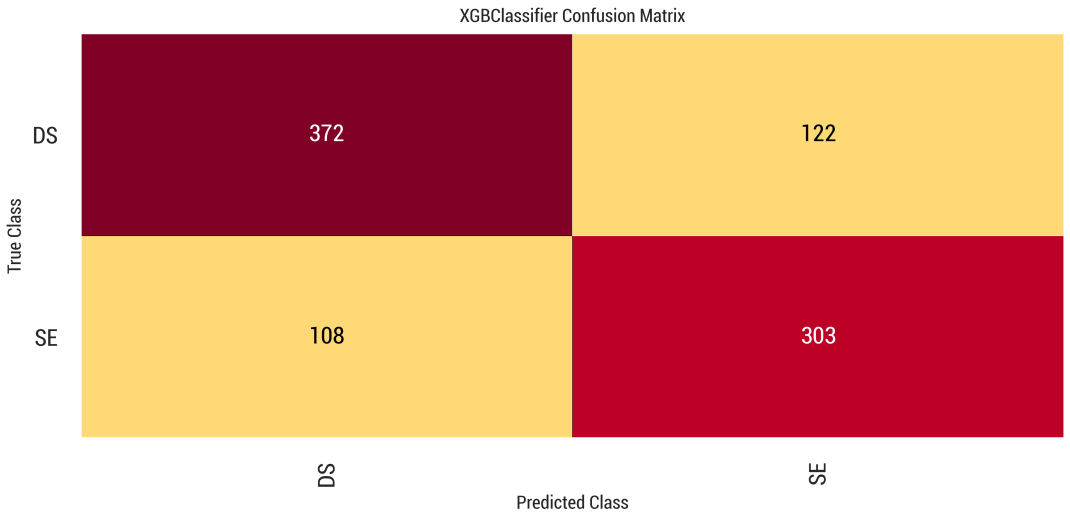


Figure 15.2: Confusion matrix

```
fig, ax = plt.subplots(figsize=(8, 4))
disp = metrics.ConfusionMatrixDisplay(confusion_matrix=cm,
                                      display_labels=['DS', 'SE'])
disp.plot(ax=ax, cmap='Blues')
```

Sometimes it is easier to understand a confusion matrix if you use fractions instead of counts. The `ORmaliizze=True` parameter will do this for us.

```
fig, ax = plt.subplots(figsize=(8, 4))
cm = metrics.confusion_matrix(y_test, xgb_def.predict(X_test),
                              normalize='true')
disp = metrics.ConfusionMatrixDisplay(confusion_matrix=cm,
                                      display_labels=['DS', 'SE'])
disp.plot(ax=ax, cmap='Blues')
```

15.3 Precision and Recall

Precision and recall are two important evaluation metrics for machine learning. *Precision* measures the proportion of correct positive predictions made by the model divided by all of the positive predictions. I like to think of this as how relevant the results are.

Recall (statisticians also like the term *sensitivity*) measures the proportion of actual positive examples that the model correctly predicts. I like to think of this as how many relevant results are returned.

Recall and precision are often at odds with each other. If you want high precision, then you can classify the sample that is the most positive as positive and get 100% precision, but you will have a low recall because the other positive samples will be mislabeled. Conversely, you can get high recall by classifying everything as positive. This will give you 100% recall but poor precision. How you want to balance this tradeoff depends on your business needs.

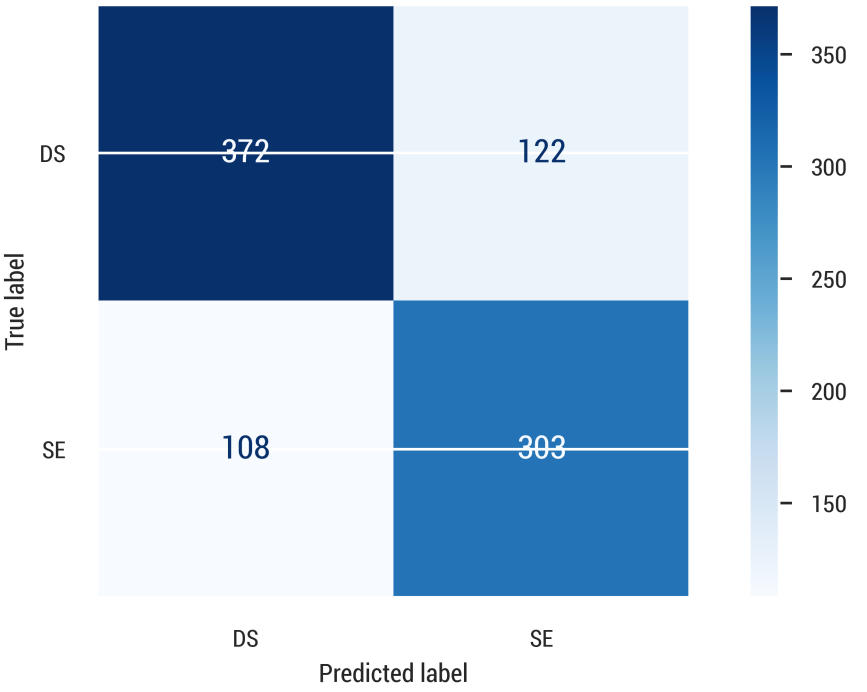


Figure 15.3: Confusion matrix from scikit learn

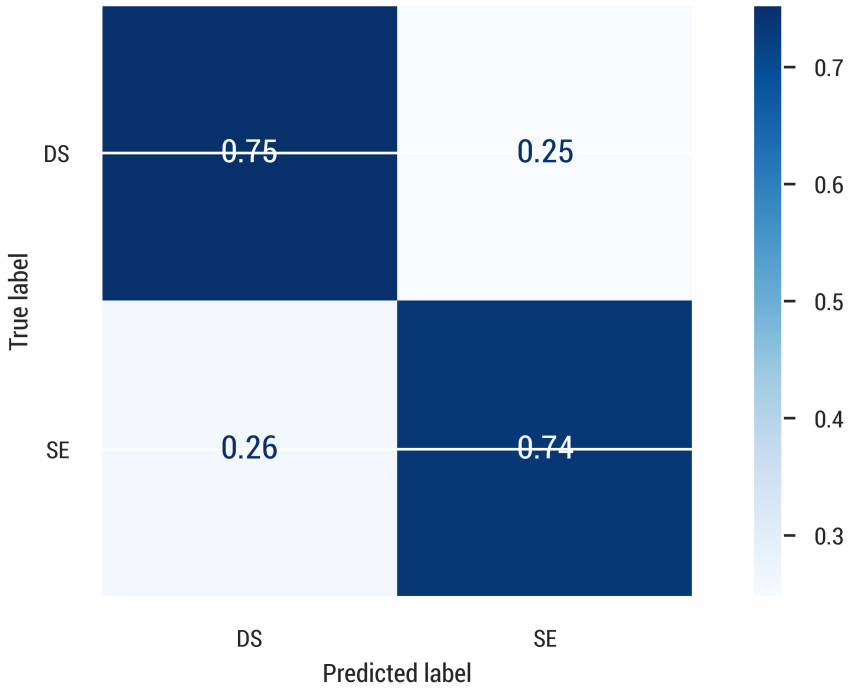


Figure 15.4: Normalized confusion matrix from scikit learn showing fractions.

You can measure the precision with the `precision_score` function from scikit-learn.

```
>>> metrics.precision_score(y_test, xgb_def.predict(X_test))
0.7129411764705882
```

You can measure the recall with the `recall_score` function from scikit-learn.

```
>>> metrics.recall_score(y_test, xgb_def.predict(X_test))
0.7372262773722628
```

To visualize the tradeoff between these two measures, you can use a *precision-recall curve* found in Yellowbrick.

```
from yellowbrick import classifier
fig, ax = plt.subplots(figsize=(8, 4))
classifier.precision_recall_curve(xgb_def, X_train, y_train,
    X_test, y_test, micro=False, macro=False, ax=ax, per_class=True)
ax.set_ylim((0,1.05))
```

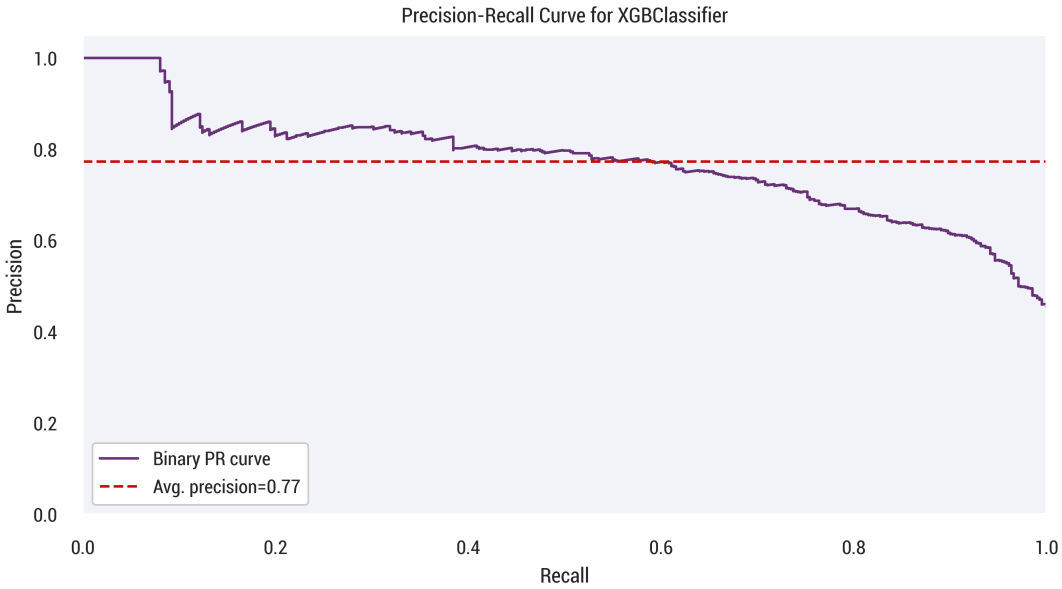


Figure 15.5: Precision-recall curve from Yellowbrick

This curve needs a little explanation. It shows the precision against the recall. That might not be very clear because we just calculated those scores, which are a single value. So, where does this plot with a line come from? Many machine learning models have a `.predict_proba` method that returns a probability for both positive and negative labels rather than just a single label or value for prediction.

One could imagine that if you changed how a model assigns a label to some threshold. If the probability of the positive label is above a threshold of .98, assign the positive label. This would likely produce a model with high precision and low recall (a point on the upper left

of our plot). You will produce the precision-recall plot if you loosen this threshold from .98 down to .01 and track both precision and recall.

This plot also shows the average precision across the threshold values and plots it as the dotted red line.

Now imagine that your boss says she does not want any false positives. If that metric is most important, you can raise the threshold (and train to optimize the `'precision'` metric when creating your model). Given such considerations, this curve can help you understand how your model might perform.

15.4 F1 Score

Another metric that might be useful is the *F1 score*. This is the harmonic mean of precision and recall. If you want to balance precision and recall, this single number can help describe the performance.

```
>>> metrics.f1_score(y_test, xgb_def.predict(X_test))
0.7248803827751197
```

Scikit-learn also has a text-based classification report. The *support* column is the number of samples for each label. The *macro average* is the average of the class behavior. The *weighted average* is based on the count (support).

```
>>> print(metrics.classification_report(y_test,
...     y_pred=xgb_def.predict(X_test), target_names=['DS', 'SE']))
              precision    recall  f1-score   support

          DS       0.78      0.75      0.76       494
          SE       0.71      0.74      0.72       411

    accuracy                           0.75       905
   macro avg       0.74      0.75      0.74       905
weighted avg       0.75      0.75      0.75       905
```

The Yellowbrick library provides a *classification report* that summarizes precision, recall, and f1-score for both positive and negative labels:

```
fig, ax = plt.subplots(figsize=(8, 4))
classifier.classification_report(xgb_def, X_train, y_train,
    X_test, y_test, classes=['DS', 'SE'],
    micro=False, macro=False, ax=ax)
```

15.5 ROC Curve

A *Receiver Operating Characteristic* curve or ROC curve is another useful plot for model diagnoses that requires some explaining.

It was first used during World War II to understand how effective radar signals were at detecting enemy aircraft. It measures the *true positive rate* (or recall) against the false positive rate (or *fallout*, false positive count divided by the count of all negatives) over a range of thresholds.

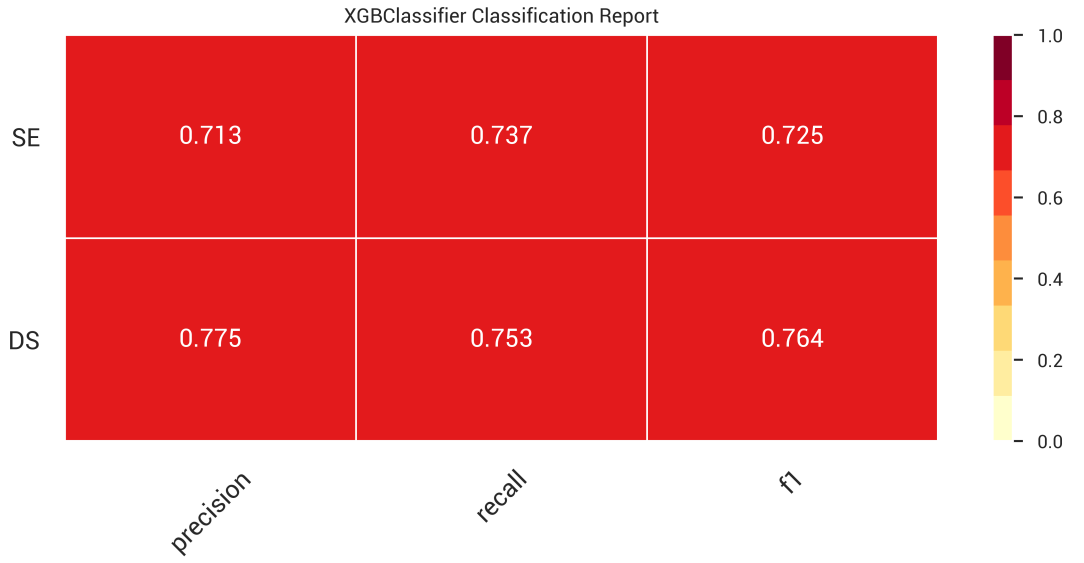


Figure 15.6: Classification report from Yellowbrick

Let's look at an example and then explain it. We will plot two ROC curves on the same plot—one from the default XGBoost model and the other from our Hyperopt-trained model. We will use the scikit learn library `RocCurveDisplay.from_estimator` method.

```
fig, ax = plt.subplots(figsize=(8,8))
metrics.RocCurveDisplay.from_estimator(xgb_def,
                       X_test, y_test,ax=ax, label='default')
metrics.RocCurveDisplay.from_estimator(xg_step,
                       X_test, y_test,ax=ax)
```

Again, imagine sweeping the probability threshold as we did for the precision-recall plot. At a high threshold, we would want a high true positive rate and a low false positive rate (if the threshold is .9, only values that the model gave a probability > .9 would be classified as positive). However, remember that the true positive rate is also the recall. A high threshold does not necessarily pull out all of the positive labels. The line starts at (0,0). As the threshold lowers, the line moves up and to the right until the threshold is low enough that we have classified everything as positive and thus have 100% recall (and 100% FPR).

Also, note that the *Area Under Curve* (AUC) is reported as .84 for our hyperopt model. Because this is a unit square, a perfect model would have an AUC of 1.0.

Folks often ask if a given AUC value is "good enough" for their model. Here's how I interpret an ROC curve. Generally, you cannot tell if a model is sufficiently good for your application from the AUC metric or an ROC plot. However, you can tell if a model is bad. If the AUC is less than .5, it indicates that the model performs worse than guessing (and you should do the opposite of what your model says).

Suppose the reported AUC is 1 (or very close to it). That usually tells me one of two things. First, the problem is such that it is very simple, and perhaps that machine learning is not needed. More often, it is the case that the modeler has *leaked data* into their model such

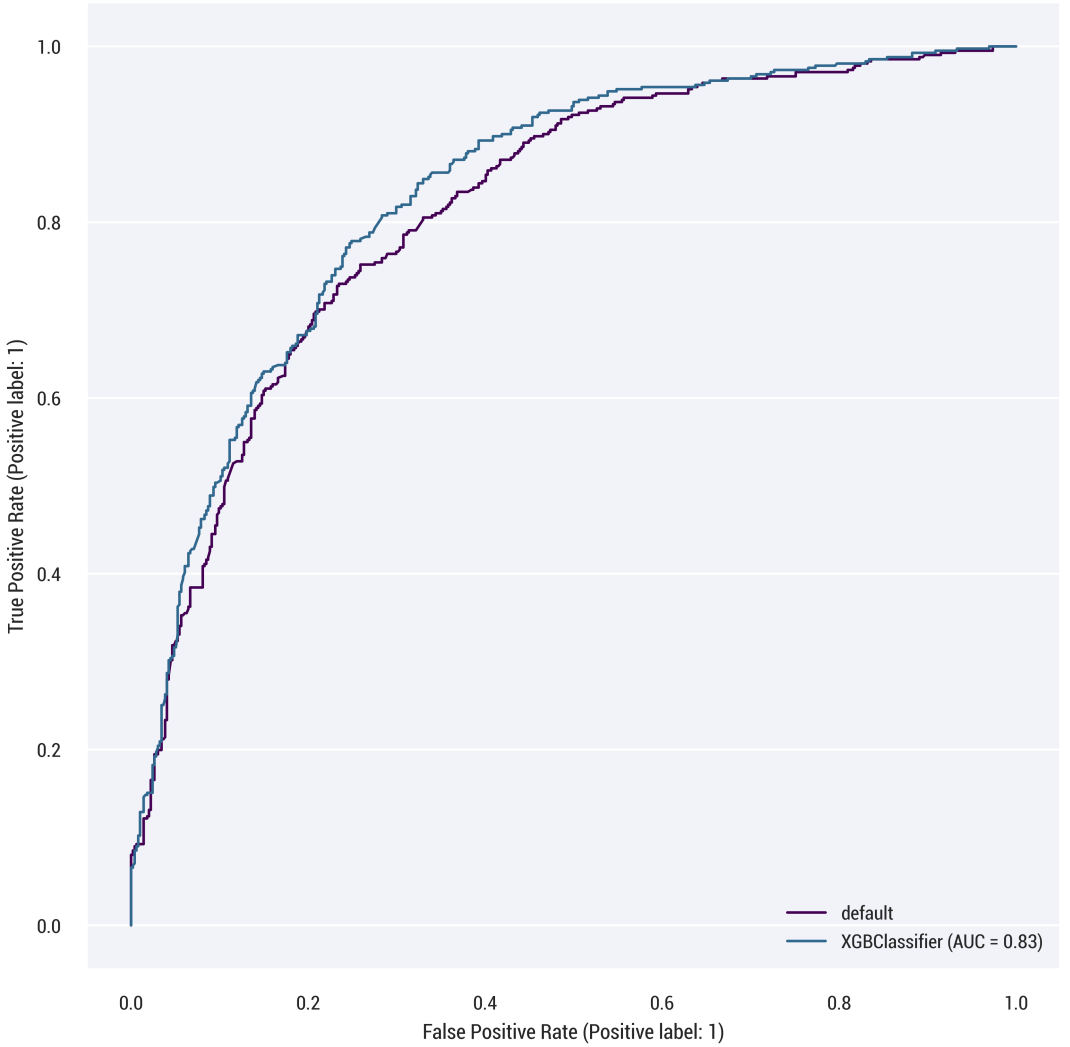


Figure 15.7: ROC Curve for out of the box model (default) and the step wise tuned model. Models that have curves above and to the left of other curves are better models. The AUC is the area under the curve. If the AUC is < .5, then the model is bad.

that it knows the future. Data leakage is when you include data in the model that wouldn't be known when you ask the model to make a prediction.

Imagine that you are building a machine learning model to predict the sentiment of movie reviews. You have a dataset of 100 reviews, where 50 of them are positive, and 50 of them are negative. You train a model that correctly predicts the sentiment of each review.

Then, you realize that your model has a serious data leakage problem. You forgot to remove the column that contains the movie's rating from the dataset, and you included this column in the model's training. This means that the model is not learning from the text of the reviews, but it is learning from the movie's rating. When you remove this feature, the accuracy of the model drops.

You can use an ROC curve to determine if a model is better than another model (at a given threshold). If one model has a higher AUC or bulges out more to the upper left corner, it is better. But being a better model is insufficient to determine whether a model would work for a given business context.

In the above plot, it appears that the default model has worse performance than the step-wise model. You should note that the step-wise model curve is to the left and above the default curve.

You can leverage the ROC curve to understand overfitting. If you plot an ROC curve for both the training and testing data, you would hope that they look similar. A good ROC curve for the training data that yields a poor testing ROC curve is an indication of overfitting.

Let's explore that with the default model and the stepwise model.

```
fig, axes = plt.subplots(figsize=(8, 4), ncols=2)
metrics.RocCurveDisplay.from_estimator(xgb_def,
                        X_train, y_train,ax=axes[0], label='detault train')
metrics.RocCurveDisplay.from_estimator(xgb_def,
                        X_test, y_test,ax=axes[0])
axes[0].set(title='ROC plots for default model')

metrics.RocCurveDisplay.from_estimator(xg_step,
                        X_train, y_train,ax=axes[1], label='step train')
metrics.RocCurveDisplay.from_estimator(xg_step,
                        X_test, y_test,ax=axes[1])
axes[1].set(title='ROC plots for stepwise model')
```

It looks like the training performance of the stepwise model is worse. However, that is because the default model appears to be overfitting, and our testing score improves with the tuning even though the training score has decreased.

15.6 Threshold Metrics

The models (in scikit-learn and XGBoost) don't expose the threshold as a hyperparameter. You can create a subclass to experiment with the threshold.

```
class ThresholdXGBClassifier(xgb.XGBClassifier):
    def __init__(self, threshold=0.5, **kwargs):
        super().__init__(**kwargs)
        self.threshold = threshold

    def predict(self, X, *args, **kwargs):
        """Predict with `threshold` applied to predicted class probabilities.
```

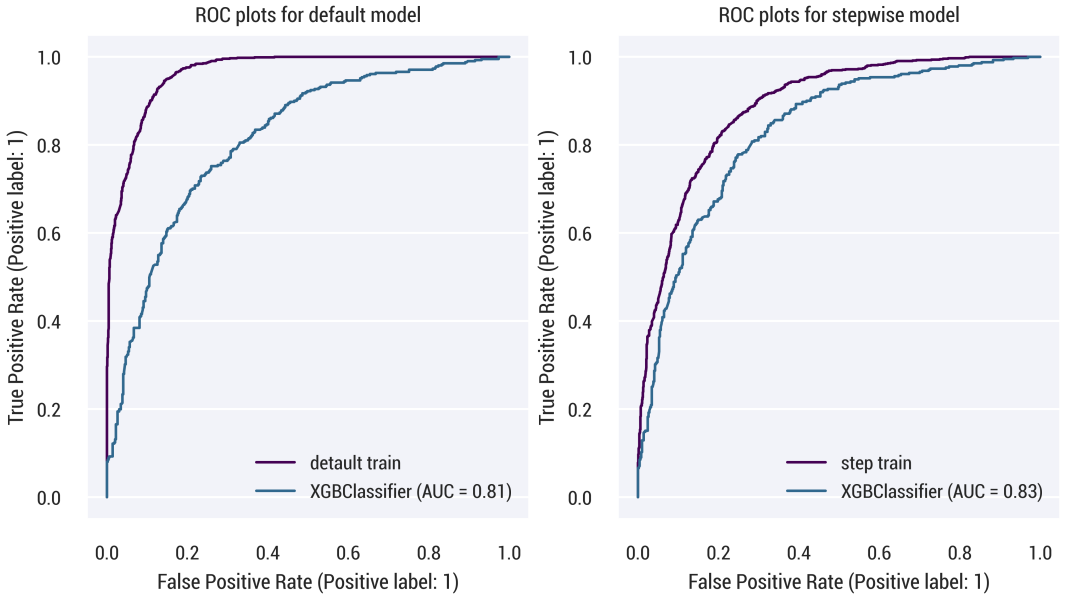


Figure 15.8: Exploring overfitting with ROC curves. On the left is the out-of-the-box model. Note that the training score is much better than the test score. With the stepwise model, the training score decreases, but the score of the test data has improved.

```
        """
        proba = self.predict_proba(X, *args, **kwargs)
        return (proba[:, 1] > self.threshold).astype(int)
```

Here is an example of using the class. Notice that the first row of the testing data has a probability of .857 of being positive.

```
>>> xgb_def = xgb.XGBClassifier()
>>> xgb_def.fit(X_train, y_train)
>>> xgb_def.predict_proba(X_test.iloc[[0]])
array([[0.14253652, 0.8574635 ]], dtype=float32)
```

When we predict this row with the default model, it comes out as positive.

```
>>> xgb_def.predict(X_test.iloc[[0]])
array([1])
```

If we set the threshold to .9, the prediction becomes negative.

```
>>> xgb90 = ThresholdXGBClassifier(threshold=.9, verbosity=0)
>>> xgb90.fit(X_train, y_train)
>>> xgb90.predict(X_test.iloc[[0]])
array([0])
```

Here is code that you can use to see if it would be appropriate to use a different threshold. This visualization shows how many common metrics respond as the threshold is changed.

```
def get_tpr_fpr(probs, y_truth):
    """
    Calculates true positive rate (TPR) and false positive rate
    (FPR) given predicted probabilities and ground truth labels.

    Parameters:
    probs (np.array): predicted probabilities of positive class
    y_truth (np.array): ground truth labels

    Returns:
    tuple: (tpr, fpr)
    """
    tp = (probs == 1) & (y_truth == 1)
    tn = (probs < 1) & (y_truth == 0)
    fp = (probs == 1) & (y_truth == 0)
    fn = (probs < 1) & (y_truth == 1)
    tpr = tp.sum() / (tp.sum() + fn.sum())
    fpr = fp.sum() / (fp.sum() + tn.sum())
    return tpr, fpr


vals = []
for thresh in np.arange(0, 1, step=.05):
    probs = xg_step.predict_proba(X_test)[:, 1]
    tpr, fpr = get_tpr_fpr(probs > thresh, y_test)
    val = [thresh, tpr, fpr]
    for metric in [metrics.accuracy_score, metrics.precision_score,
                   metrics.recall_score, metrics.f1_score,
                   metrics.roc_auc_score]:
        val.append(metric(y_test, probs > thresh))
    vals.append(val)

fig, ax = plt.subplots(figsize=(8, 4))
(pd.DataFrame(vals, columns=['thresh', 'tpr/rec', 'fpr', 'acc',
                             'prec', 'rec', 'f1', 'auc'])
 .drop(columns='rec')
 .set_index('thresh')
 .plot(ax=ax, title='Threshold Metrics')
)
```

15.7 Cumulative Gains Curve

This curve is useful for maximizing marketing response rates (evaluating a model when you have finite resources). It plots the *gain* (the recall or sensitivity) against the ordered samples. The baseline is what a random prediction would give you. The gain is calculated as the true positive rate if you were to order the predictions by the probability against the percentage of samples.

Look at the plot below from the `scikitplot` library. I have augmented it to show the optimal gains (what a perfect model would give you).

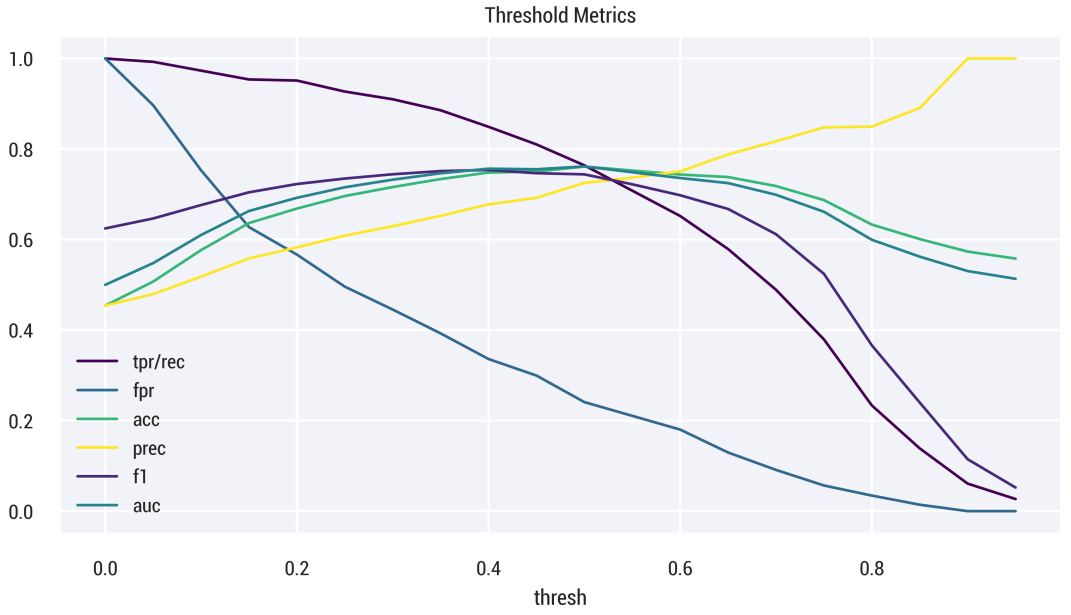


Figure 15.9: Threshold plots

```
import scikitplot
fig, ax = plt.subplots(figsize=(8, 4))
y_probs = xgb_def.predict_proba(X_test)
scikitplot.metrics.plot_cumulative_gain(y_test, y_probs, ax=ax)
ax.plot([0, (y_test == 1).mean(), 1], [0, 1, 1], label='Optimal Class 1')
ax.set_ylim(0, 1.05)
ax.annotate('Reach 60% of\nClass 1\nby contacting top 35%', xy=(.35, .6),
            xytext=(.55,.25), arrowprops={'color':'k'})
ax.legend()
```

Here is an example of reading the plot. If you want to reach 60% of the positive audience, you can trace the .6 from the y-axis to the plot. It indicates that you need to contact the top 35% of the sample if you want to reach that number. Note that if we had a perfect model, we would only need to reach out to the 45% of them in the positive class (see the blue line).

15.8 Lift Curves

A lift curve is a plot showing the true positive rate and the cumulative positive rate of a machine learning model as the prediction threshold varies. It leverages the same x-axis from the cumulative gains plot, but the y-axis is the ratio of gains to randomly choosing a label. It indicates how much better the model is than guessing.

If you only can reach the top 20% of your audience (move up from .2 on the x-axis), you should be almost 1.8x better than randomly choosing labels.

I have also augmented this with the optimal model.

```
fig, ax = plt.subplots(figsize=(8, 4))
y_probs = xgb_def.predict_proba(X_test)
```

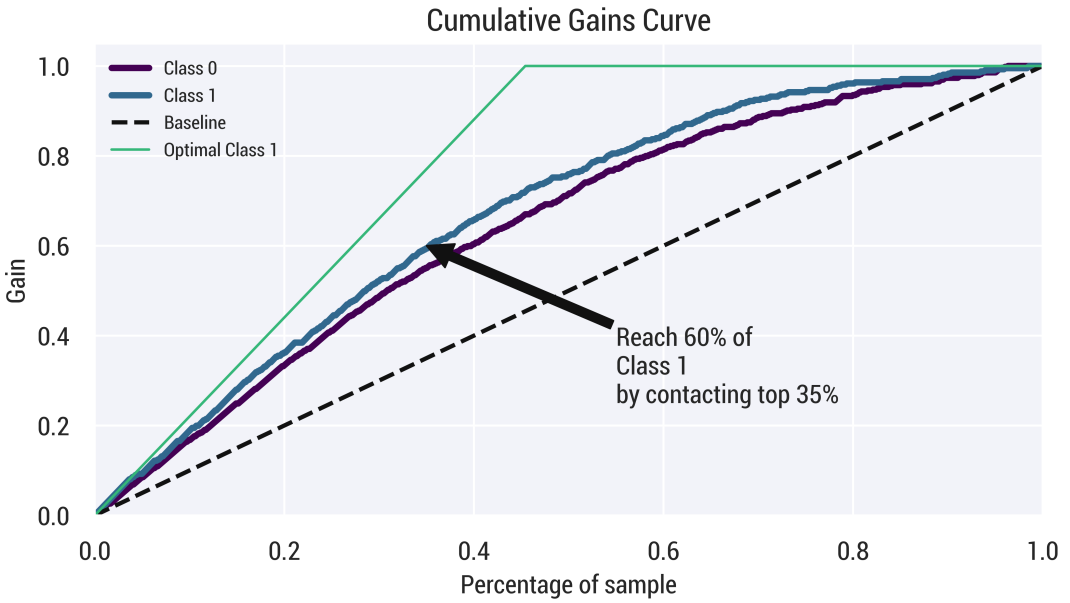


Figure 15.10: Cumulative gains plot

```
scikitplot.metrics.plot_lift_curve(y_test, y_probs, ax=ax)
mean = (y_test == 1).mean()
ax.plot([0, mean, 1], [1/mean, 1/mean, 1], label='Optimal Class 1')
ax.legend()
```

15.9 Summary

In this chapter, we explored various metrics and plots to evaluate a classification model. There is no one size fits all metric or plot to understand a model. Many times it depends on the needs of the business.

Precision, recall, accuracy, and F1 are common and important metrics for evaluating and comparing the performance of a machine learning model. ROC curves, precision-recall curves, and lift plots are visual representations of the performance of a machine learning model. These metrics and plots provide valuable and complementary information about the performance of a machine learning model, and they are widely used for evaluating, comparing, and diagnosing the performance of the model. They are important and useful tools for improving, optimizing, and maximizing the performance of the model, and they are essential for understanding and interpreting the results of the model.

15.10 Exercises

1. How can cross-validation be used to evaluate the performance of a machine learning model?
2. What is the difference between precision and recall in evaluating a machine learning model?

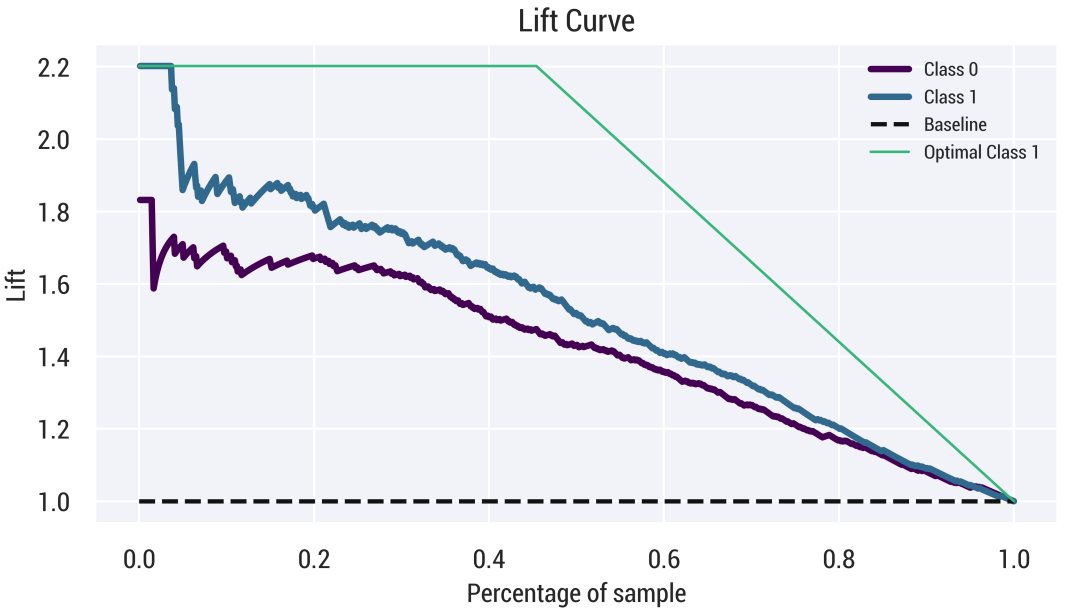


Figure 15.11: Lift Curve for out of the box model

3. What is the F1 score, and how is it used to evaluate a machine learning model?
4. In what situations might it be more appropriate to use precision, recall, accuracy, or F1 as a metric to evaluate the model's performance?
5. What is the purpose of a ROC curve in evaluating a machine learning model?
6. How can ROC curves, precision-recall curves, and lift plots be used to improve, optimize, and maximize the performance of a machine learning model?

Chapter 16

Training For Different Metrics

We just looked at different metrics for model evaluation. In this chapter, we will look at training a model to maximize those metrics.

16.1 Metric overview

We will compare optimizing a model with the metrics of accuracy, the area under the ROC curve. Accuracy is a measure of how often the model correctly predicts the target class. Precision is a measure of how often the model is correct when it predicts the positive class. Recall is a measure of how often the model can correctly identify the positive class. The area under the ROC curve is one measure to balance precision and recall. The F1 score is another metric that combines both by using the harmonic mean of precision and recall.

16.2 Training with Validation Curves

Let's use a validation curve to visualize tuning the `learning_rate` for both accuracy and area under ROC. (Note that you can get a high recall by just classifying everything as positive, so that is not a particularly interesting metric to maximize.) We will use the Yellowbrick library. Remember that a validation curve tracks the performance of the metric as we sweep through hyperparameter values.

The `validation_curve` function accepts a `scoring` parameter to determine a metric to track. We will pass in `scoring='accuracy'` to the first curve and `scoring='roc_auc'` to the second.

```
from yellowbrick import model_selection as ms
fig, ax = plt.subplots(figsize=(8, 4))
ms.validation_curve(xgb.XGBClassifier(), X_train, y_train,
    scoring='accuracy', param_name='learning_rate',
    param_range=[0.001, .01, .05, .1, .2, .5, .9, 1], ax=ax
)
ax.set_xlabel('Accuracy')
```

```
fig, ax = plt.subplots(figsize=(8, 4))
ms.validation_curve(xgb.XGBClassifier(), X_train, y_train,
    scoring='roc_auc', param_name='learning_rate',
    param_range=[0.001, .01, .05, .1, .2, .5, .9, 1], ax=ax
    )
ax.set_xlabel('roc_auc')
```

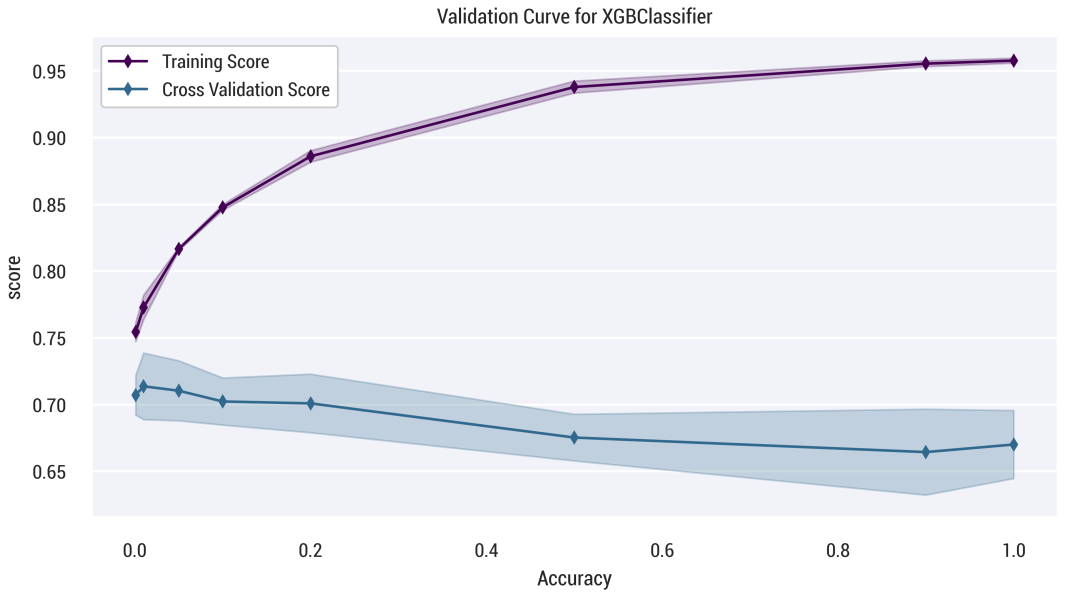


Figure 16.1: Validation Curve for XGBoost model tuning learning_rate for accuracy

You can see that the optimal learning rate for accuracy is around .01 (given our limited range of choices). The shape of the area under the ROC curve is similar but shifted up, and the maximum cross-validation score is around .05. In general, tuning for different metrics will reveal different hyperparameter values.

16.3 Step-wise Recall Tuning

Let's use the `hyperopt` library to optimize AUC. Let's pass in `roc_auc_score` for the `metric` parameter.

```
from sklearn.metrics import roc_auc_score
from hyperopt import hp, Trials, fmin, tpe
params = {'random_state': 42}

rounds = [{'max_depth': hp.quniform('max_depth', 1, 9, 1),  # tree
           'min_child_weight': hp.loguniform('min_child_weight', -2, 3)},
          {'subsample': hp.uniform('subsample', 0.5, 1),   # stochastic
           'colsample_bytree': hp.uniform('colsample_bytree', 0.5, 1)},
          {'gamma': hp.loguniform('gamma', -10, 10)}, # regularization
          {'learning_rate': hp.loguniform('learning_rate', -7, 0)} # boosting
]

for round in rounds:
    params = {**params, **round}
    trials = Trials()
    best = fmin(fn=lambda space: xhelp.hyperparameter_tuning(
        space, X_train, y_train, X_test, y_test, metric=roc_auc_score),
        space=params,
```

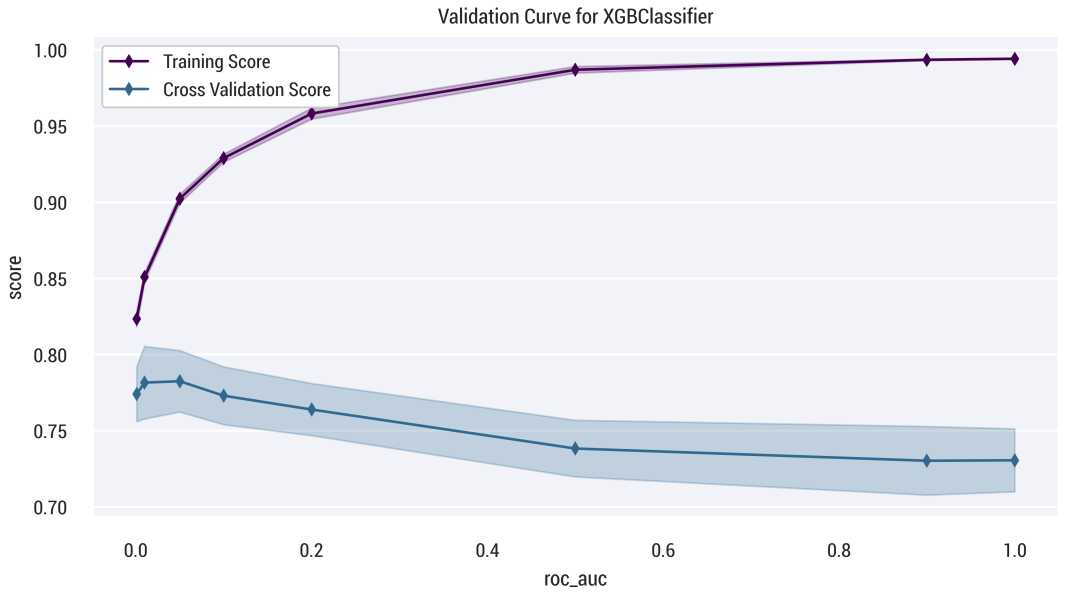


Figure 16.2: Validation Curve for XGBoost model tuning learning_rate for area under the ROC curve

```
    algo=tpe.suggest,
    max_evals=40,
    trials=trials,
)
params = {**params, **best}
```

Now let's compare the default model for recall versus our optimized model.

```
>>> xgb_def = xgb.XGBClassifier()
>>> xgb_def.fit(X_train, y_train)
>>> metrics.roc_auc_score(y_test, xgb_def.predict(X_test))
0.7451313573096131

>>> # the values from above training
>>> params = {'random_state': 42,
...  'max_depth': 4,
...  'min_child_weight': 4.808561584650579,
...  'subsample': 0.9265505972233746,
...  'colsample_bytree': 0.9870944989347749,
...  'gamma': 0.1383762861356536,
...  'learning_rate': 0.13664139307301595}
```

Below I use the `verbose=100` parameter to only show the output every 100 iterations.

```
>>> xgb_tuned = xgb.XGBClassifier(**params, early_stopping_rounds=50,
...    n_estimators=500)
>>> xgb_tuned.fit(X_train, y_train, eval_set=[(X_train, y_train),
...     (X_test, y_test)], verbose=100)
```

```
[0] validation_0-logloss:0.66207    validation_1-logloss:0.66289
[100]   validation_0-logloss:0.44945    validation_1-logloss:0.49416
[150]   validation_0-logloss:0.43196    validation_1-logloss:0.49833
XGBClassifier(base_score=0.5, booster='gbtree', callbacks=None,
              colsample_bylevel=1, colsample_bynode=1,
              colsample_bytree=0.9870944989347749, early_stopping_rounds=50,
              enable_categorical=False, eval_metric=None, feature_types=None,
              gamma=0.1383762861356536, gpu_id=-1, grow_policy='depthwise',
              importance_type=None, interaction_constraints='',
              learning_rate=0.13664139307301595, max_bin=256,
              max_cat_threshold=64, max_cat_to_onehot=4, max_delta_step=0,
              max_depth=4, max_leaves=0, min_child_weight=4.808561584650579,
              missing=nan, monotone_constraints='()', n_estimators=500,
              n_jobs=0, num_parallel_tree=1, predictor='auto', random_state=42, ...)
```

It looks like our tuned model has improved the area under the ROC curve score.

```
>>> metrics.roc_auc_score(y_test, xgb_tuned.predict(X_test))
0.7629510328319394
```

16.4 Summary

In this chapter, we explored optimizing models around different metrics. Talk to the business owners and determine what metric is important for you to optimize. Then tune your model to that metric. The default metric of accuracy is a fine default, but it might not be what you need.

16.5 Exercises

1. What is the purpose of a validation curve?
2. How does tuning a model for recall differ from tuning for accuracy?
3. How does the metric choice influence a model's hyperparameters?
4. Can a model be optimized for both precision and recall simultaneously? Discuss.
5. In what situations might it be important to prioritize recall over precision or vice versa?

Chapter 17

Model Interpretation

Consider a bank offering loans. The bank wants to be able to tell a user why they were rejected. Saying, "We are denying your application, but if you deposit $2,000 into your account, we will approve it." is better than saying, "We deny your application. Sorry, we can't tell you why because our model doesn't give us that information." The company might value explainability more than performance.

Data scientists classify models as *white box* and *black box* models. A white box machine learning model is a model that is transparent and interpretable. This means that the model's workings are completely understood and can be explained in terms of the input features and the mathematical operations used to make predictions.

In contrast, a black box machine learning model is a model that is not transparent or interpretable. This means that the model's workings are not easily understood, and it is difficult or impossible to explain how the model makes predictions.

A decision tree model is a white box model. You can explain exactly what the model is doing.

An XGBoost model is considered a black box model. I would probably call it a grey model (or maybe dark grey). We can explain what happens, but we need to sift through multiple trees to come to an answer. It is unlikely that a user of the model is going to want to track through 587 trees to determine why it made a prediction.

This chapter will explore different models and how to interpret them. We will also discuss the pros and cons of the interpretation techniques.

17.1 Logistic Regression Interpretation

Let's start with a Logistic Regression model. Logistic regression uses a logistic function to predict the probability that a given input belongs to a particular class. Like linear regression, it learns coefficients for each feature. These coefficients are multiplied by the feature values and summed to make a prediction. It is a white box model because the result comes from the match in the previous sentence. In fact, you can use these coefficients to determine how features impact the model (by looking at the coefficient's sign) and the impact's magnitude (by looking at the magnitude of the coefficient, assuming that the features are standardized).

To aid with the interpretation of the coefficients, we *standardize* the features before running logistic regression. Standardization is a technique used to transform data with a mean of zero and a standard deviation of one. Standardization is helpful for machine learning because it helps ensure that all features are on a similar scale, which can improve the performance or behavior of certain models, such as linear regression, logistic regression, support vector machines, and neural networks.

Let's look at how this model works with our data (we set `penalty=None` to turn off the regularization of this model):

```
>>> from sklearn import linear_model, preprocessing
>>> std = preprocessing.StandardScaler()
>>> lr = linear_model.LogisticRegression(penalty=None)
>>> lr.fit(std.fit_transform(X_train), y_train)
>>> lr.score(std.transform(X_test), y_test)
0.7337016574585635
```

In this case, logistic regression gives us a similar score to XGBoost's default model. If I had data that performed the same with both logistic regression and XGBoost, I would use the logistic regression model because it is simpler. Given two equivalent results, I will choose the simpler version. Logistic regression also is a white box model and is easy to explain.

Let's look at the `.coef_` attribute. These are the feature coefficients or weights that we learned during the fitting of the model. (This uses scikit learn's convention of adding an underscore to the attribute learned while training the model.)

```
>>> lr.coef_
array([[-1.56018160e-01, -4.01817103e-01,  6.01542610e-01,
        -1.45213121e-01, -8.13849902e-02, -6.03727624e-01,
         3.11683777e-02,  3.16120596e-02, -3.14510213e-02,
        -4.59272439e-04, -8.21683100e-03, -5.27737710e-02,
        -4.48524110e-03,  1.01853988e-01,  3.49376790e-01,
        -1.79149729e-01,  2.41389081e-02, -3.37424750e-01]])
```

This is a NumPy array and a little hard to interpret on its own. I prefer to stick this in a Pandas series and add the corresponding column names to the values. You can make a bar plot to visualize them.

```
fig, ax = plt.subplots(figsize=(8, 4))
(pd.Series(lr.coef_[0], index=X_train.columns)
 .sort_values()
 .plot.barh(ax=ax)
)
```

The wider the bar, the higher the impact of the feature. Positive values push towards the positive label, in our case Software Engineer. Negative labels push towards the negative label, Data Scientist. The years of experience column correlates with software engineering, and using the R language correlates with data science. Also, the *Q1_Prefer not to say* feature does not have much impact on this model.

Given this result, I might try to make an even simpler model. A model that only considers features that have an absolute magnitude above 0.2.

17.2 Decision Tree Interpretation

A decision tree is also a white box model. I will train a decision tree with a depth of seven and explore the *feature importance* results. These values are learned from fitting the model, and similar to coefficients in the logistic regression model, they give us some insight into how the features impact the model.

```
>>> tree7 = tree.DecisionTreeClassifier(max_depth=7)
>>> tree7.fit(X_train, y_train)
>>> tree7.score(X_test, y_test)
0.7337016574585635
```

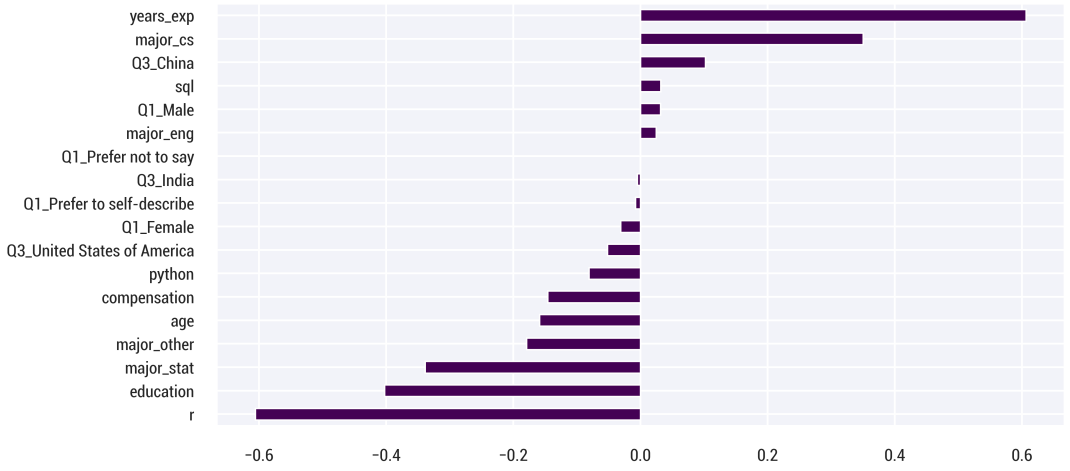


Figure 17.1: Logistic Regression feature coefficients

Let's inspect the `.feature_importances_` attribute. This is an array of values that indicate how important each feature is in making predictions using a decision tree model. It is learned during the call to `.fit` (hence the trailing underscore). Like logistic regression, it gives a magnitude of importance. In scikit-learn, it is calculated from the decrease in Gini importance when making a split on a feature. Unlike logistic regression, it does not show the direction (this is because a tree, unlike logistic regression, can capture non-linear relationships and use a feature to direct results toward both positive and negative labels).

```
fig, ax = plt.subplots(figsize=(8, 4))
(pd.Series(tree7.feature_importances_, index=X_train.columns)
 .sort_values()
 .plot.barh(ax=ax)
)
```

It is interesting to note that the feature importances of the decision tree are not necessarily the same as the coefficients of the logistic regression model.

I will plot the first three levels of the tree to compare the nodes to the feature importance scores.

```
import dtreeviz
dt3 = tree.DecisionTreeClassifier(max_depth=3)
dt3.fit(X_train, y_train)

viz = dtreeviz.model(dt3, X_train=X_train, y_train=y_train,
    feature_names=list(X_train.columns), target_name='Job',
    class_names=['DS', 'SE'])
viz.view()
```

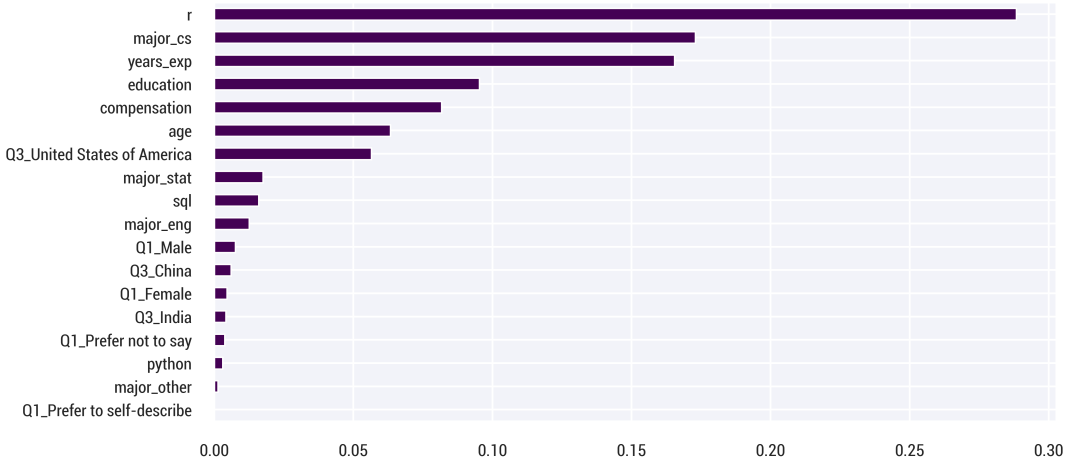


Figure 17.2: Decision tree feature importance

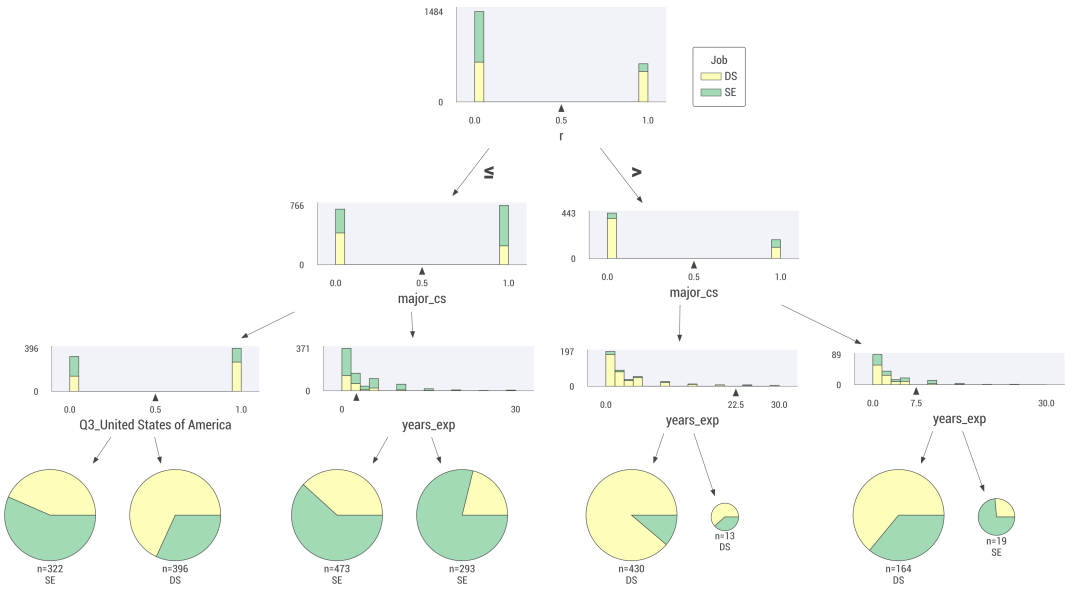


Figure 17.3: Decision tree visualization using dtreeviz.

17.3 XGBoost Feature Importance

The XGBoost model also reports feature importance. The `.feature_importances_` attribute reports the normalized gain across all the trees when the feature is used.

```
xgb_def = xgb.XGBClassifier()
xgb_def.fit(X_train, y_train)

fig, ax = plt.subplots(figsize=(8, 4))
(pd.Series(xgb_def.feature_importances_, index=X_train.columns)
 .sort_values()
 .plot.barh(ax=ax)
)
```

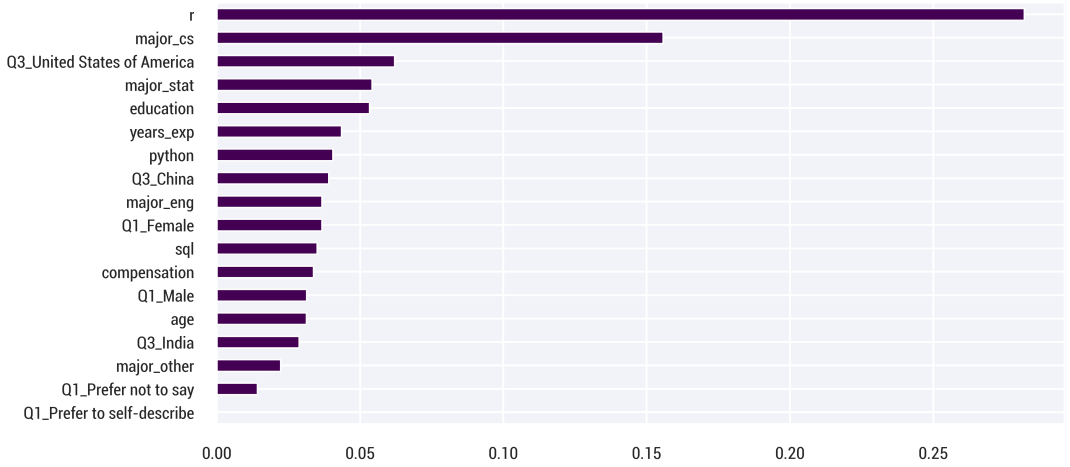


Figure 17.4: XGBoost default feature importance

You can also visualize the feature importance directly from XGBoost. The `.plot_importance` method has a `importance_type` parameter to change how importance is measured.

- The `'gain'` feature importance is calculated as the total gain in the model's performance that results from using a feature.
- The `'weight'` feature importance is calculated as the number of times a feature is used in the model.
- The `'cover'` feature importance is calculated as the number of samples that are affected by a feature.

```
fig, ax = plt.subplots(figsize=(8, 4))
xgb.plot_importance(xgb_def, importance_type='cover', ax=ax)
```

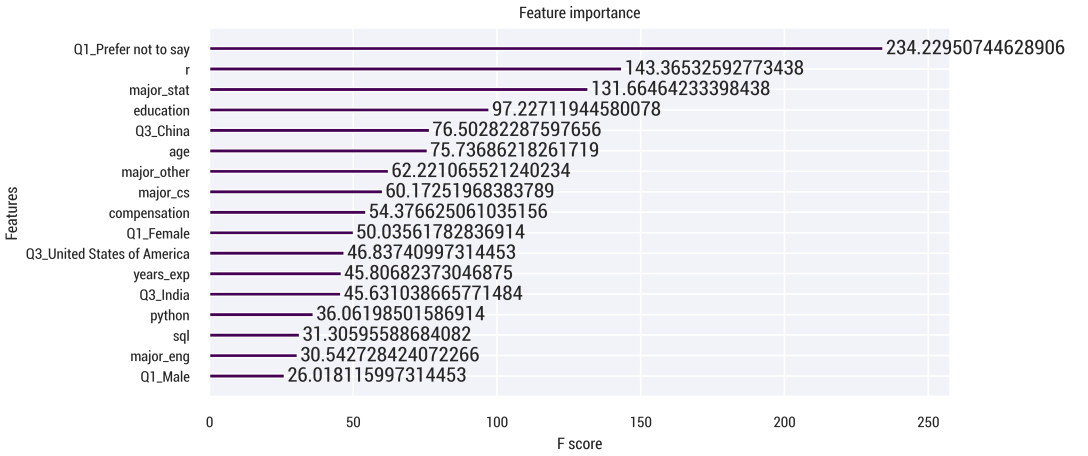


Figure 17.5: XGBoost feature importance for cover importance

17.4 Surrogate Models

Another way to tease apart the XGBoost model is to train a decision tree to its predictions and then explore the interpretable decision tree. Let's do that.

```
from sklearn import tree

sur_reg_sk = tree.DecisionTreeRegressor(max_depth=4)
sur_reg_sk.fit(X_train, xgb_def.predict_proba(X_train)[:,-1])
```

Let's export the tree to examine it. If you want to make a tree that goes from left to right with scikit-learn, you need to use the `export_graphviz` function.

Note

I ran the following code to convert the DOT file to a PNG.

```
tree.export_graphviz(sur_reg_sk, out_file='img/sur-sk.dot',
    feature_names=X_train.columns, filled=True, rotate=True,
    fontname='Roboto Condensed')
```

You can use the `dot` command to generate the PNG:

```
dot -Gdpi=300 -Tpng -oimg/sur-sk.png img/sur-sk.dot  # HIDE
```

This surrogate model can provide insight into interactions. Nodes that split on a different feature than a parent node often have an interaction. It looks like *R* has an interaction with *major_cs*. Also, the *years_exp* and *education* columns seem to interact. We will explore those in the SHAP chapter.

You can also learn about non-linearities with surrogate models. When the same node feature follows itself, that often indicates a non-linear relationship with the target.

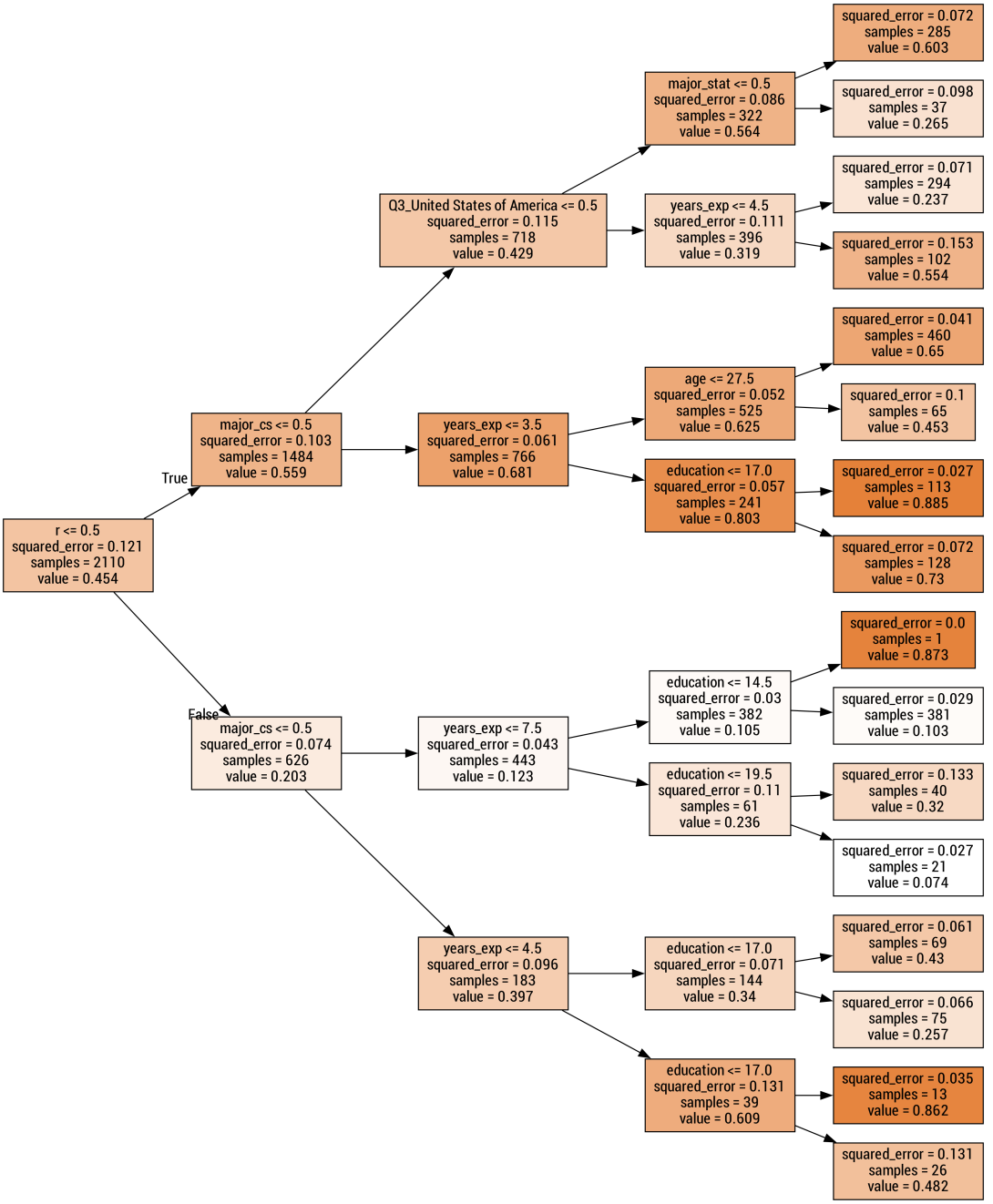


Figure 17.6: A surrogate tree of the model. You can look for the whitest and orangest nodes and trace the path to understand the prediction.

17.5 Summary

Interpretability in machine learning refers to understanding how a model makes predictions. A model is considered interpretable if it can explain its workings regarding the input features and the mathematical operations that the model uses to make predictions.

Interpretability is important because it allows us to understand how a model arrives at its predictions and identify any potential biases or errors. This can be useful for improving the model or for explaining the model's predictions to decision-makers.

Feature importance in XGBoost measures each feature's importance in making predictions using a decision tree model. XGBoost provides three feature importances: weight, gain, and cover. These feature importances are calculated differently and can provide different insights about the model.

17.6 Exercises

1. What is the difference between white box and black box models regarding interpretation?
2. How is logistic regression used to make predictions and how can the coefficients be interpreted to understand the model's decision-making process?
3. How can decision trees be interpreted, and how do they visualize the model's decision-making process?
4. What are some potential limitations of using interpretation techniques to understand the decision-making process of a machine learning model?
5. How can the feature importance attribute be used to interpret the decision-making process of a black box model?
6. In what situations might it be more appropriate to use a white box model over a black box model, and vice versa?

Chapter 18

xgbfir (Feature Interactions Reshaped)

In this chapter, we will explore how features relate to each other and the labels. We will define feature interactions, show how to find them in XGBoost, and then introduce interaction constraints to see if it improves our model.

18.1 Feature Interactions

When you have multiple columns in a machine-learning model, you may want to understand how those columns *interact* with each other. Statisticians say that X_1 and X_2 interact when the X_1 changes are not constant in the change to y but also depend on X_2.

You could represent this as a formula when using linear or logistic regression. You can create a new column that multiplies the two columns together:

$y = aX_1 + bX_2 + cX_1X_2$

In decision trees, interaction is a little different. A feature, X_1 does not interact with another feature, X_2, if X_2 is not in the path, or if X_2 is used the same in each path. Otherwise, there is an interaction, and both features combined to impact the result.

18.2 xgbfir

The `xgbfir` library will allow us to find the top interactions from our data in an XGBoost model. You will have to install this third-party library and also the `openpyxl` library (a library to read Excel files) to explore the feature interactions.

I'm not going to lie. The interface of this library is a little odd. You create a model, then run the `saveXgbFI` function, and it saves an Excel file with multiple sheets with the results. Let's try it out:

```
import xgbfir
xgbfir.saveXgbFI(xgb_def, feature_names=X_train.columns, OutputXlsxFile='fir.xlsx')
```

Let's read the first sheet. This sheet lists all of the features from our data. It includes a bunch of other columns that include metric values and the rank of the metrics. The final order of the rows is the average rank of the metrics.

Here are the columns:

- *Gain*: Total gain of each feature or feature interaction
- *FScore*: Amount of possible splits taken on a feature or feature Interaction

- *wFScore*: Amount of possible splits taken on a feature or feature interaction weighted by the probability of the splits to take place
- *Average wFScore*: wFScore divided by FScore
- *Average Gain*: Gain divided by FScore
- *Expected Gain*: Total gain of each feature or feature interaction weighted by the probability of the gain

For each of the above columns, another column shows the rank. Finally, there is the *Average Rank*, the *Average Tree Index*, and the *Average Tree Depth* columns.

Let's sort the features by the *Average Rank* column:

```
>>> fir = pd.read_excel('fir.xlsx')
>>> print(fir
...  .sort_values(by='Average Rank')
...  .head()
...  .round(1)
... )
    Interaction   Gain  FScore  ...  Average Rank  Average Tree Index  \
2             r  517.8      84  ...           3.3                44.6
0     years_exp  597.0     627  ...           4.5                45.1
5     education  296.0     254  ...           4.5                45.2
1  compensation  518.5     702  ...           4.8                47.5
4      major_cs  327.1      96  ...           5.5                48.9

   Average Tree Depth
2                 2.6
0                 3.7
5                 3.3
1                 3.7
4                 3.6

[5 rows x 16 columns]
```

This looks like the *R* and *years_exp* features are important for our model.

However, this chapter is about interactions, and this isn't showing interactions, just a single column. So we need to move on to the other sheets found in the Excel file.

The *Interaction Depth 1* sheet shows how pairs of columns interact. Let's look at it:

```
>>> print(pd.read_excel('fir.xlsx', sheet_name='Interaction Depth 1').iloc[:20]
...     .sort_values(by='Average Rank')
...     .head(10)
...     .round(1)
... )
               Interaction    Gain  FScore  wFScore  Average wFScore  \
1      education|years_exp   523.8     106     14.8              0.1
0              major_cs|r  1210.8      15      5.4              0.4
6   compensation|education   207.2     103     18.8              0.2
11          age|education   133.2      80     27.2              0.3
3       major_cs|years_exp   441.3      36      4.8              0.1
```

```
5           age|years_exp   316.3     216     43.9             0.2
4         age|compensation   344.7     219     38.8             0.2
15    major_stat|years_exp    97.7      32      6.7             0.2
14             education|r   116.5      14      4.6             0.3
18                 age|age    90.5      66     24.7             0.4

    Average Gain  Expected Gain  Gain Rank  FScore Rank  wFScore Rank  \
1            4.9           77.9          2           5             8
0           80.7          607.6          1          45            20
6            2.0           34.0          7           6             7
11           1.7           25.6         12           8             4
3           12.3          108.2          4          20            25
5            1.5           44.0          6           3             1
4            1.6           30.6          5           2             2
15           3.1           20.4         16          25            15
14           8.3           72.3         15          52            27
18           1.4           16.6         19          11             6

    Avg wFScore Rank  Avg Gain Rank  Expected Gain Rank  Average Rank  \
1                 43              8                   3          11.5
0                  8              1                   1          12.7
6                 32             25                   9          14.3
11                12             40                  13          14.8
3                 46              3                   2          16.7
5                 26             57                   7          16.7
4                 34             48                  11          17.0
15                24             14                  14          18.0
14                13              5                   4          19.3
18                 7             62                  16          20.2

    Average Tree Index  Average Tree Depth
1                 38.0                 3.5
0                 12.3                 1.6
6                 50.6                 3.7
11                38.8                 3.6
3                 29.2                 3.2
5                 45.6                 3.9
4                 48.9                 3.9
15                25.5                 3.1
14                40.4                 2.4
18                48.0                 3.6
```

This tells us that *education* is often followed by *years_exp* and *major_cs* is often followed by *R*. This hints that these features might have an impact on each other. Let's explore that a little more by exploring the relationship between the features.

We can create a correlation heatmap with both Pandas and Seaborn. Here's the pandas version which is useful inside of Jupyter.

```
(X_train
 .assign(software_eng=y_train)
 .corr(method='spearman')
```

```
 .loc[:, ['education', 'years_exp', 'major_cs', 'r', 'compensation', 'age']]
 .style
 .background_gradient(cmap='RdBu', vmin=-1, vmax=1)
 .format('{:.2f}')
)

import seaborn as sns
fig, ax = plt.subplots(figsize=(8, 4))
sns.heatmap(X_train
            .assign(software_eng=y_train)
            .corr(method='spearman')
            .loc[:, ['age','education', 'years_exp', 'compensation', 'r',
                     'major_cs', 'software_eng']],
            cmap='RdBu', annot=True, fmt='.2f', vmin=-1, vmax=1, ax=ax
)
```

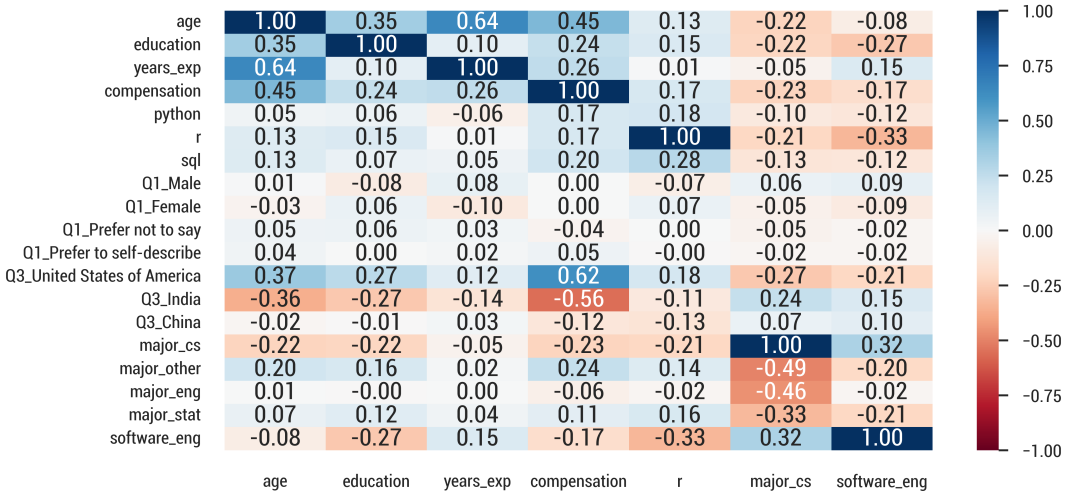


Figure 18.1: Correlation heatmap between subset of features

Interestingly, the correlation between *years_exp* and *education* is close to 0.10. There may be a non-linear relationship that a tree model can tease apart. Let's explore that with a scatter plot. We will use Seaborn to add a fit line and color the plot based on the label.

```
import seaborn.objects as so
fig = plt.figure(figsize=(8, 4))
(so
 .Plot(X_train.assign(software_eng=y_train), x='years_exp', y='education',
       color='software_eng')
 .add(so.Dots(alpha=.9, pointsize=2), so.Jitter(x=.7, y=1))
 .add(so.Line(), so.PolyFit())
 .scale(color='viridis')
 .on(fig)  # not required unless saving to image
 .plot()   # ditto
```

)

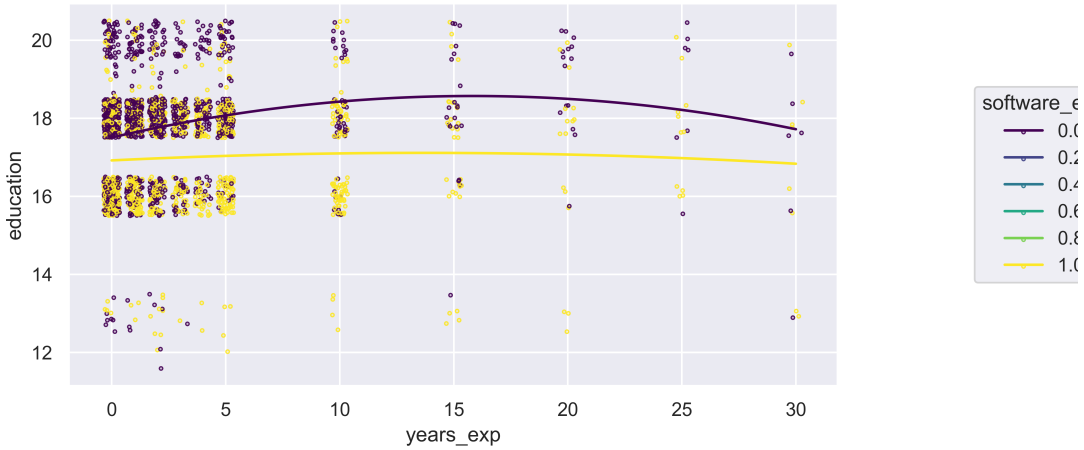


Figure 18.2: Scatter plot of education vs years_exp

It looks like the education levels for software engineers are lower than for data scientists.

Now, let's explore the relationships between *major_cs* and *R* and the target label. This has three dimensions of categorical data. We can use Pandas grouping or the `pd.crosstab` shortcut to quantify this:

```
>>> print(X_train
...   .assign(software_eng=y_train)
...   .groupby(['software_eng', 'r', 'major_cs'])
...   .age
...   .count()
...   .unstack()
...   .unstack()
... )
major_cs        0         1
r               0    1    0    1
software_eng
0             410  390  243  110
1             308   53  523   73
```

This shows that if the user didn't use R, they have a higher probability of being a software engineer.

Here is the `pd.crosstab` version:

```
>>> both = (X_train
...   .assign(software_eng=y_train)
... )
>>> print(pd.crosstab(index=both.software_eng, columns=[both.major_cs, both.r]))
major_cs        0         1
r               0    1    0    1
```

```
software_eng
0             410  390  243  110
1             308   53  523   73
```

We can also visualize this. Here is a visualization of this data using a slope graph. My interpretation of this data is that not using R is a strong indication of the label "software engineer". Here is the interaction between using R and not studying CS. It is an indicator of the "data science" label. However, studying CS and using R is not quite so clear. You can see why the tree might want to look at the interaction between these features rather than just looking at them independently.

This plot is a little more involved than most in this book, but I think it illustrates the point well.

```
fig, ax = plt.subplots(figsize=(8, 4))
grey = '#999999'
blue = '#16a2c6'
font = 'Roboto'

data = (X_train
 .assign(software_eng=y_train)
 .groupby(['software_eng', 'r', 'major_cs'])
 .age
 .count()
 .unstack()
 .unstack())

(data
 .pipe(lambda adf: adf.iloc[:,-2:].plot(color=[grey,blue], linewidth=4, ax=ax,
                                        legend=None) and adf)
 .plot(color=[grey, blue, grey, blue], ax=ax, legend=None)
)

ax.set_xticks([0, 1], ['Data Scientist', 'Software Engineer'], font=font, size=12,
              weight=600)
ax.set_yticks([])
ax.set_xlabel('')
ax.text(x=0, y=.93, s="Count Data Scientist or Software Engineer by R/CS",
        transform=fig.transFigure, ha='left', font=font, fontsize=10, weight=1000)
ax.text(x=0, y=.83, s="(Studied CS) Thick lines\n(R) Blue", transform=fig.transFigure,
        ha='left', font=font, fontsize=10, weight=300)
for side in 'left,top,right,bottom'.split(','):
    ax.spines[side].set_visible(False)
# labels
for left,txt in zip(data.iloc[0], ['Other/No R', 'Other/R', 'CS/No R', 'CS/R']):
    ax.text(x=-.02, y=left, s=f'{txt} ({left})', ha='right', va='center',
            font=font, weight=300)
for right,txt in zip(data.iloc[1], ['Other/No R', 'Other/R', 'CS/No R', 'CS/R']):
    ax.text(x=1.02, y=right, s=f'{txt} ({right})', ha='left', va='center',
            font=font, weight=300)
```

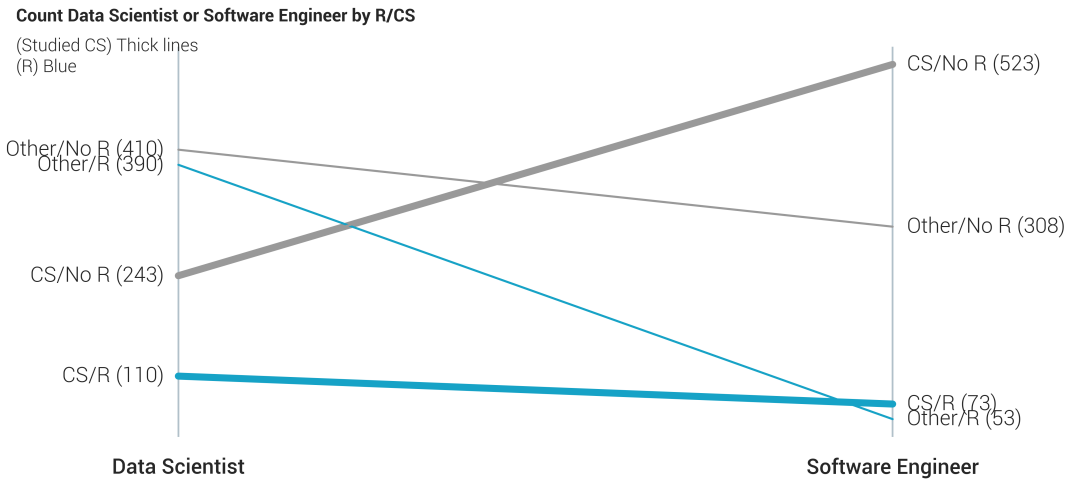


Figure 18.3: Slopegraph of R vs study CS

18.3 Deeper Interactions

You can get interactions with three columns from the *Interaction Depth 2* sheet.

```
>>> print(pd.read_excel('fir.xlsx', sheet_name='Interaction Depth 2').iloc[:20]
...   .sort_values(by='Average Rank')
...   .head(5)
... )
                        Interaction         Gain  FScore  ...  Average Rank  \
0                major_cs|r|years_exp  1842.711375      17  ...     12.000000
7             age|education|years_exp   267.537987      53  ...     15.666667
13         age|compensation|education   154.313245      55  ...     15.833333
2    compensation|education|years_exp   431.541357      91  ...     17.166667
14              education|r|years_exp   145.534591      17  ...     19.000000

    Average Tree Index  Average Tree Depth
0             2.588235            2.117647
7            31.452830            3.981132
13           47.381818            3.800000
2            47.175824            4.010989
14           34.352941            2.588235

[5 rows x 16 columns]
```

I'm not going to explore these here. Again, these interactions can lead to insights about how various features relate to each other that would be hard to come by without this analysis.

18.4 Specifying Feature Interactions

The XGBoost library can limit feature interactions. You can specify a list of features that can only be in trees with other limited features. This could lead to models that are more robust, simpler or comply with regulations.

Let's train models with limited interactions and a default model and see how the performance compares.

I will take the top entries from the interactions and put them in a nested list. Each entry in this list shows which groups of columns are able to follow other columns in a given tree. For example, *education* is in the first, third, and fourth lists. This means that *education* can be followed by *years_exp*, *compensation*, or *age* columns, but not the remaining columns.

```
constraints = [['education', 'years_exp'], ['major_cs', 'r'],
   ['compensation', 'education'], ['age', 'education'],
   ['major_cs', 'years_exp'], ['age', 'years_exp'],
   ['age', 'compensation'], ['major_stat', 'years_exp'],
]
```

I'll write some code to filter out the columns that I want to use, so I can feed data with just those columns into my model.

```
def flatten(seq):
    res = []
    for sub in seq:
        res.extend(sub)
    return res


small_cols = sorted(set(flatten(constraints)))
```

```
>>> print(small_cols)
['age', 'compensation', 'education', 'major_cs', 'major_stat', 'r', 'years_exp']
```

```
>>> xg_constraints = xgb.XGBClassifier(interaction_constraints=constraints)
>>> xg_constraints.fit(X_train.loc[:, small_cols], y_train)
>>> xg_constraints.score(X_test.loc[:, small_cols], y_test)
```

```
0.7259668508287292
```

It looks like this model does ok when it uses 7 of the 18 features. But it does not exhibit better performance than the default model (.745) trained on all of the data.

Here is a visualization of the first tree. You can see that the *r* column is only following by *major_cs* in this tree.

```
my_dot_export(xg_constraints, num_trees=0, filename='img/constrains0_xg.dot',
              title='First Constrained Tree')
```

18.5 Summary

In this chapter, we explored feature interactions. Exploring feature interactions can give you a better sense for how the columns in the data impact and relate to each other.

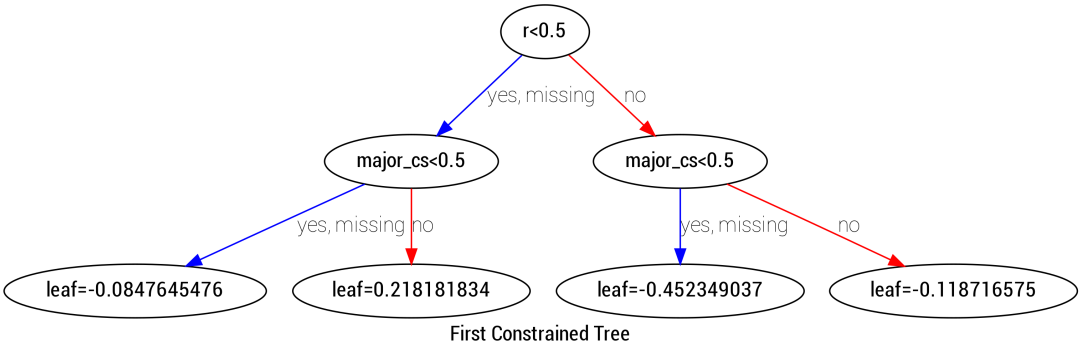


Figure 18.4: First constrained tree

18.6 Exercises

1. What is an interaction in a machine learning model?
2. What is the xgbfir library used for in machine learning?
3. How is the Average Rank column calculated in the xgbfir library?
4. What is the purpose of the Interaction Depth 1 sheet in the xgbfir library?
5. How can the xgbfir library be used to find the top feature interactions in an XGBoost model?
6. What is the purpose of using interaction constraints in a machine learning model?
7. What are the potential drawbacks of using interaction constraints in a machine learning model?

Chapter 19

Exploring SHAP

Have you ever wanted to peek inside the "black box" of a machine learning model and see how it's making predictions? While feature importance ranks the features, it doesn't indicate the direction of the impact.

Importance scores also can't model non-linear relationships. For example, when I worked with solid-state storage devices, there was a "bathtub curve" relationship between usage and lifetime. Devices with little usage often had infant mortality issues, but once the device had survived "burn-in", they often lasted until end-of-life failure. A feature importance score cannot help to tell this story. However, using a tool like SHAP can.

The `shap` library is a useful tool for explaining black box models (and white box models too). *SHapley Additive Explanations* (SHAP) is a mechanism that explains a model's global and local behavior. At a global level, SHAP can provide a rank order of feature importance and how the feature impacts the results. It can model non-linear relationships. At the local level of an individual prediction, SHAP can explain how each feature contributes to the final prediction.

19.1 SHAP

SHAP works with both classification and regression models. SHAP analyzes a model and predictions from it and outputs a value for every feature of an example. The values are determined using game theory and indicate how to distribute attribution among the features to the target. In the case of classification, these values add up to the log odds probability of the positive label. (For regression cases, SHAP values sum to the target prediction.)

One of the key advantages of the SHAP algorithm is that it can provide explanations for both white box models, where the internal workings of the model are known, and black box models, where the inner workings of the model are not known. It can also handle complex models and interactions between features and has been shown to provide accurate and consistent explanations in various settings.

Let's look at using this library with our step-tuned model. Make sure you have installed the library as it does not come with Python.

```
step_params = {'random_state': 42,
 'max_depth': 5,
 'min_child_weight': 0.6411044640540848,
 'subsample': 0.9492383155577023,
 'colsample_bytree': 0.6235721099295888,
 'gamma': 0.00011273797329538491,
 'learning_rate': 0.24399020050740935}
xg_step = xgb.XGBClassifier(**step_params, early_stopping_rounds=50,
```